Medidata Rave®

Version 5.6.3

Web Services Inbound v1.0.2

User Guide

Medidata Solutions, Inc.
Corporate Office
79 Fifth Avenue
New York, N.Y. 10003
+1 212 918 1800

Document Version 3.1
December 2009

Disclaimer: Given that Rave allows users to translate and change text strings, any screens shown in this document are only a representation of the product and may not match the actual interface you are currently using.

Medidata Rave® 5.6.2 is certified by the Clinical Data Interchange Standards Consortium (CDISC) for Operational Data Modeling (ODM) standards ODM v1.2 and ODM v1.2.1.

Table of Contents

Table of Figures

About This Guide

This document describes how customers, vendors, sponsors and partners can use Rave Web Services Inbound (RWSI) V1.0.2 to exchange CDISC Operational Data Model (ODM) standard XML over HTTP web services.

Note: As it contains hyperlinks, this guide is designed to be read as a PDF and not a printed document.

Audience

This document is aimed at users of RWSI and those responsible for specifying and configuring RWSI implementations.

Organization

1. About this Guide.

This chapter contains general information required to begin using this guide.

2. Introduction to Rave Web Services Inbound

This chapter contains introductory and background information on RWSI.

3. Using RWSI to Update Clinical Data

This chapter includes information about how to call RWSI and some helpful steps to assist in the resolution of any problems which may be encountered with RWSI.

4. Metadata Import Scenarios

This chapter contains some typical examples of ODM XML metadata for import using RWSI. These examples are based on the Mediflex study which is available to registered users at https://innovate.mdsol.com.

5. Metadata Export Scenarios

This chapter contains some typical examples of ODM XML metadata for export using RWSI. These examples are based on the Mediflex study which is available to registered users at https://innovate.mdsol.com.

6. Clinical Data Import Scenarios

This chapter contains some typical examples of ODM XML clinical data for import using RWSI. These examples are based on the Mediflex study which is available to registered users at https://innovate.mdsol.com.

7. Appendix A

This section contains a list of all RWSI response codes and descriptions returned following an import transaction.

8. Appendix B

This section contains snippets of code in different coding languages to illustrate how RWSI could be called to initiate an ODM data import. This includes samples for C#, Erlang, Java, Python and Ruby.

9. Appendix C

List of Urls and descriptions

10. Glossary of Terms

11. Revision History

References

The following document is referred to in this document.

Document Name	Version
Rave Web Services Inbound v1.0. API Quick Reference.doc (available on MyMedidata.com)	Current

Conventions used in this document

The following is a list of the typographical conventions used in this document:

`Courier New`

Used for any code examples, file and directory names, program and command names.

Boldface

Used for email addresses and pathnames, URLs, and for emphasizing new terms.

Introduction to Rave Web Services Inbound

Overview

Rave Web Services Inbound v1.0 (RWSI) integrates Medidata Rave with third party systems to exchange CDISC ODM v1.3 standard clinical data or metadata with Rave synchronously and with immediate confirmation. This document describes how to use RWSI for this purpose.

Features

RWSI is a validated Rave Add-On. Version 1.0.2 supports:

- Clinical data import
- Transaction logging
- Error reporting
- Study design (Metadata) import
- Study design (Metadata) export.

System Compatibility and Software Limitations

RWSI 1.0 is compatible with Rave version 5.6.3 or higher on all X86 platforms, Windows Operating systems (2000, XP, NT), and SQL Server 2005.

Error Reporting

For every transaction, RWSI returns a response, and, if there is an error, an error description and error location in Xpath notation. See Appendix A for more information on response codes and 'Appendix B' for Sample Code for Calling RWSI.

RWSI checks the request XML in two ways: by checking that it is valid XML and by checking it against the ODM v1.3 schema (with backward compatibility support to ODM 1.2 for clinical data only).

The CDISC ODM 1.3 schema used for validation is available on the CDISC ODM website:

http://www.cdisc.org/

In the 'STANDARDS' menu, select 'Operational Data Model'.

For more information on how to correctly construct the request ODM v1.3 XML, see the 'Rave Web Services Inbound v1.0 API Quick Reference' guide.

Security

By default, RWSI accepts transactions over HTTPS, but it can be configured to allow non-encrypted HTTP transactions for use within internal networks.

The external calling system must include the username and password information in the HTTP(S) request using HTTP basic authentication. RWSI connects to Rave using the username and password information provided in the inbound message.

Rave Role Permissions

RWSI uses the Rave Security model to determine what actions are permitted and what data can be accessed. The Rave user account used by the external system calling RWSI must have an associated role which includes the relevant permissions. The minimum privilege an

associated role must include for clinical data entry is the 'Entry' action and the role should also be set up with form and field restrictions for the relevant studies.

The minimum role permissions RWSI requires for users to perform all clinical data operations are the following:

CanInactivateForm, CanReactivateForm, CreateSubject, ModifyPrimaryRecord, SubjectAdmin, UsePrimarySubjectName, ViewSubjectsWithAnyStatus.

For descriptions of the roles and permissions, refer to the latest version of the Medidata Rave Configuration: Workflow and Interface Settings Manual.

Roles created after RWSI installation will not be available for RWSI until the RWSI cache is flushed (by default the cache will be refreshed every 24 hours). If the user requires the newly created roles to be available immediately, use the following URL to clear the cache.

https://<Rave URL>/ <RWS directory>/WebService.aspx?CacheFlush

For example,

https://innovate.mdsol.com/RaveWebServices/WebService.aspx?CacheFlush

Subjects

Clinical data relates to a specified subject, and the studies and subjects that a given user can see relate to the security access granted to them by their role associations in Rave. In order to be able to create a subject (including adding the default matrix), the Rave user account must have the 'CreateSubject' permission assigned to at least one of its associated roles.

Subjects cannot be removed from Rave by RWSI. If RWSI receives a request to remove a subject, it will return an error 'Removing a subject is not permitted' in its response.

Folders (Instances)

The user account needs to have the 'SubjectAdmin' permission assigned to at least one of its associated roles in order to be able to create a folder (instance).

Forms (Data Pages)

The user account needs to have the 'SubjectAdmin' permission assigned to at least one of its associated roles in order to be able to create a form (data page).

See the 'Rave Web Services Inbound v 1.0 API Quick Reference' guide for more information on the error response codes associated with subjects, folders and forms.

Using RWSI to Update Clinical Data

Getting Started

In order to use RWSI to update clinical data in Rave, an XML document compatible with the CDISC ODM v1.3 schema must be created and submitted to RWSI.

RWSI provides immediate confirmation as to whether the request has been successful or whether it has failed.

Note:
There is no restriction on how the ODM XML document is sent to RWSI, any application capable of posting ODM XML to a target URL can be used. See 'Appendix B' for more information.

The ODM XML document should be sent to the target URL as the body of an HTTP POST request. For instance, for the Rave URL 'https://innovate.mdsol.com/' the document should be posted to the following location:

https://innovate.mdsol.com/RaveWebServices/WebService.aspx?PostODMClinicalData

The content type of the document must be set as 'text/xml'. Documents should be encoded as UTF-8.

In general

https://<Rave URL>/<RWS directory>/WebService.aspx?PostODMClinicalData

<RWS directory>

Is the Web directory where RWSI is installed, which usually will be "RaveWebServices". If a different name is being used then the Medidata Project Manager will provide the RWS directory name.

Note: RequestContentTypeAllowed
RWS checks the incoming request to make sure the Content-Type is allowable. This is to help protect from malicious posts.

Requests have limits in the number of bytes. The table below gives the default limit of bytes by request type. An administrator can change these values in the database table

`WebServicesRequestProcessors`

Type of Request	Default Maximum Size (bytes)
ClinicalData POST	1000000

Type of Request	Default Maximum Size (bytes)
Metadata GET	20000000
Metadata POST	20000000

Example Requests and Responses

An example valid request for posting clinical data

Below is an example of a valid XML document submitted in a request to RWSI. Beneath that is the response returned through RWSI indicating that the request has been successful.

```
<ODM xmlns='http://www.cdisc.org/ns/odm/v1.3' ODMVersion='1.3'
FileType='Transactional' FileOID='Example-2' CreationDateTime='2008-01-
01T00:00:00'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='123 ABC' TransactionType='Update'>
      <SiteRef LocationOID='4567'/>
      <StudyEventData StudyEventOID='SCREEN'>
        <FormData FormOID='DM'>
          <ItemGroupData ItemGroupOID='DM'>
            <ItemData ItemOID='BRTHDTC' Value='01 JAN 1980'/>
            <ItemData ItemOID='SEX' Value='MALE'/>
            <ItemData ItemOID='RACE' Value='3'/>
            <ItemData ItemOID='COUNTRY' Value='GBR'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>
```

Note:

Notice that in the example valid request the environment is defined in the StudyOID within brackets. The StudyOID **must** exactly match the project name as it appears in Rave with the environment in brackets appended, e.g. 'Mediflex (Dev)' (with a space between the project name and environment), unless it is for the production environment, in which case only the project name is required.

See the 'Rave Web Services Inbound v1.0 API Quick Reference' guide for more information on the StudyOID and how to correctly construct the request ODM XML.

An example successful response for posting clinical data

RWSI responses are not in CDISC ODM format (since CDISC ODM does not include response information definitions).

```
<Response ReferenceNumber="aacbaffe-8eed-45d7-bfe3-4916f4935af4"
InboundODMFileOID="Example-2" IsTransactionSuccessful="1"
SuccessStatistics="Rave objects touched: Subjects=0; Folders=0; Forms=0;
Fields=4; LogLines=0" NewRecords=""></Response>
```

Response Entry	Description
ReferenceNumber	A 256 bit alphanumeric unique identifier for the transaction
InboundODMFileOID	The FileOID of the ODM file sent to Web Services
IsTransactionSuccessful	1 = Yes, 0 = No
SuccessStatistics	This shows how many Rave objects have been 'touched', i.e. created, updated or removed. This can be compared against expectations to check if the import has worked as desired
NewRecords	Confirms insertions of log lines, and provides details of the inserted repeat key.

Important:

If any issues cannot be resolved, when contacting Medidata for support please have all of the information in the table above available to help identify the transaction and the possible causes of the issue.

If the response does not contain a ReferenceNumber, please specify the URL, Subject, Study details and details of the transaction along with the XML and username.

Note:

For clinical data: RWSI can handle more than one concurrent import. The number of concurrent imports is limited by the configuration of the Medidata web server.

For metadata: RWSI can accept more than one concurrent import request but each request will be processed sequentially in the sequence they appear "FIFO (first in first out)".

Note:

The use of 'ItemGroupRepeatKey="LAST"' allows the user to avoid the need to find and enter the sequence number of the inserted repeat key.

An example invalid request

Below is an example of a request submitted to RWSI. Beneath that is the response returned through RWSI indicating that the request has failed. This includes further information about

the nature of this failure. In the event of an error, the import transaction is rolled back and no changes are made to clinical data in the Rave database.

```
<ODM xmlns='http://www.cdisc.org/ns/odm/v1.2' ODMVersion='1.2'
FileType='Transactional' FileOID='Example-2' CreationDateTime='2008-01-
01T00:00:00'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='123 ABC' TransactionType='Update'>
      <SiteRef LocationOID='4567'/>
      <StudyEventData StudyEventOID='SCREEN'>
        <FormData FormOID='DM'>
          <ItemGroupData ItemGroupOID='DM'>
            <ItemData ItemOID='BIRTHDATE' Value='01 JAN 1980'/>
            <ItemData ItemOID='SEX' Value='MALE'/>
            <ItemData ItemOID='RACE' Value='3'/>
            <ItemData ItemOID='COUNTRY' Value='GBR'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>
```

An example failure response

RWSI responses are not in CDISC ODM format.

```
<Response ReferenceNumber="7127e3d0-92e3-4448-97b2-d6338de897f2"
InboundODMFileOID="Example-2.xml" IsTransactionSuccessful="0"
ReasonCode="RWS00041"
ErrorOriginLocation="/ODM/ClinicalData[1]/SubjectData[1]/StudyEventData[1]/F
ormData[1]/ItemGroupData[1]/ItemData[1]" SuccessStatistics="Rave objects
touched: Subjects=0; Folders=0; Forms=0; Fields=0; LogLines=0"
ErrorClientResponseMessage="Field does not exist."></Response>
```

Response Entry	Description
ReferenceNumber	A 256 bit alphanumeric unique identifier for the transaction
InboundODMFileOID	The FileOID of the ODM file client sent to RWSI
IsTransactionSuccessful	1 = Yes, 0 = No
ReasonCode	Error response code, e.g. RWS00041. See 'Appendix A' for more information on response codes
ErrorOriginLocation	Xpath notation indicating the location in the XML, submitted by the client, where the problem occurred
SuccessStatistics	This shows how many Rave objects have been 'touched', i.e. created, updated or removed. This can be compared against expectations to check if the import

Response Entry	Description
	has worked as desired. This will always be '0' in all error cases as the transaction is rolled back
ErrorClientResponseMessage	A description of the error. See 'Appendix A' for more information on response codes

Important:

If any issues cannot be resolved, when contacting Medidata please have all of the information in the table above available to help identify the transaction and the possible causes of the issue.

If the response does not contain a ReferenceNumber, please specify the URL, Subject, Study details and details of the transaction along with the XML and username.

Note: One Subject Per Transaction

RWSI only supports one subject per transaction. If data for more than one Subject is included in the ODM XML request, RWS will return an error.

Note: Audit Trails and Edit Checks

Following a successful import, automated Rave processes such as audit trails and edit checks will occur as if the imported clinical data had been directly entered into Rave.

Note: Coded Data Values

For the ItemData element Value attribute, where the value is associated to a data dictionary, the Coded Data value **must** be used. For example, '1' for male or '2' for female.

Note: Specify Values

Where fields have a data dictionary and the entry requires a Specify value, e.g. 'Other (specify)' the field is marked as non-conformant in Rave. ODM does not provide the facility to import Specify values. RWSI does not return an error response code for this.

Note: Lab

Do not use RWSI for entering data on fields associated with Rave's Lab Administration module, the current version of Batch Uploader should be used to perform this function. RWSI will not associate inserted or updated datapoints with Lab Ranges.

Note: Change Codes

The role used for RWSI should only have one Rave change code. RWSI will use the first change code associated with a user's role in Rave. If the user's role does not have a change code, no change code will be used.

Any changes made to the change code will not be reflected in Rave until the RWSI service has been restarted by Medidata. If a change code is changed after RWSI installation, please ask Medidata to restart the service. If this is not done, RWSI will return the error 'Web Service currently unavailable'. This incorrect error message reported in this situation is a known issue and will be fixed in a subsequent release of the software. To view instructions for cache flush, refer to the Getting Started section in this document.

Metadata Import Scenarios

This section contains typical examples of ODM XML metadata for import into Rave using RWSI and shows the results of these imports in Rave. These examples are based on the Mediflex study which is available to registered users at https://innovate.mdsol.com, and they include:

- Importing Metadata (study design)
- Importing extension attributes
- Importing lab settings.

Importing metadata (study design)

To create or update a study design in Rave Architect, post a valid ODM 1.3 document containing a single MetaDataVersion element to the following URL:

https://<Rave URL>/RaveWebServices/metadata/studies/<study name>/drafts

For example, for the “Mediflex” study:

https://innovate.mdsol.com/RaveWebServices/metadata/studies/Mediflex/drafts

The Rave account you use must have permissions to upload a draft.

The following sample ODM XML shows you how to import metadata into Rave to create or update a study design. The metadata elements in this example consist of these:

- The StudyName is “Mediflex”;
- The ProtocolName is “Mediflex”;
- The MetaDataVersion OID defines a draft study called “2” in project “Mediflex”;
- The StudyEventDef creates a folder, with OID “SCREEN” and name “Screen”;
- The FormDef OID “DM” creates a form, with OID “DM” and name “Demographic”;
- The ItemDef OID “DM.BRTHDAT” creates a field OID “BRTHDAT”, name “BRTHDAT” and label “Date of birth”.

See the sample ODM below:

```
<ODM FileType=”Snapshot” Granularity=”Metadata”
  CreationDateTime=”2009-05-19T14:56:39.194-00:00”
  FileOID=”PostMetaDataStudyImportDraft” ODMVersion=”1.3”
  xmlns=”http://www.cdisc.org/ns/odm/v1.3”
  xmlns:mdsol=”http://www.mdsol.com/ns/odm/metadata”>
  <Study OID=”Mediflex”>
    <GlobalVariables>
      <StudyName>Mediflex</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Mediflex</ProtocolName>
    </GlobalVariables>
    <MetaDataVersion OID=”10” Name=”2” mdsol:SignaturePrompt=”Please sign”>
      <Protocol>
        <StudyEventRef StudyEventOID=”SCREEN” Mandatory=”No” />
      </Protocol>
```

```
      <StudyEventDef OID="SCREEN" Name="Screen" Type="Common"
        Repeating="Yes">
        <FormRef FormOID="DM" Mandatory="No" />
</StudyEventDef>
<FormDef OID="DM" Name="Demographic" Repeating="No"
mdsol:OrderNumber="1" mdsol:SignatureRequired="No">
<ItemGroupRef ItemGroupOID="DM" Mandatory="Yes" />
</FormDef>
      <ItemGroupDef OID="DM" Name="DM" Repeating="No">
        <ItemRef ItemOID="DM.BRTHDAT" Mandatory="No" />
      </ItemGroupDef>
      <ItemDef OID="DM.BRTHDAT" Name="BRTHDAT" DataType="date"
        mdsol:Active="Yes" mdsol:ControlType="DateTime"
        <Question>
          <TranslatedText xml:lang="en">Date of birth</TranslatedText>
        </Question>
      </ItemDef>
    </MetaDataVersion>
  </Study>
</ODM>
```

Errors in study design identified by Architect Loader will be returned within the RWSI response.

Importing extension attributes

To import extension attributes to an existing study design in Rave Architect, post ODM XML for an existing draft with field level vendor attributes to the following URL:

https://<Rave URL>/RaveWebServices/metadata/studies/<study name>/versions/<version number>/attributes

For example, for the "Mediflex" study:

https://innovate.mdsol.com/RaveWebServices/metadata/studies/Mediflex/versions/2/attributes

The following sample ODM XML shows you how to import metadata into Rave to create or update a study design. The metadata elements in this example consist of these:

- The StudyName is "Mediflex";
- The ProtocolName is "Mediflex";
- The MetaDataVersion OID defines a draft study called "2" in project "Mediflex";
- The StudyEventDef creates a folder, with OID "SCREEN", with Name "Screen";
- The FormDef OID "DM" creates a form, with Name "Demographic";

- The Attribute Namespace is “My Integration”, with Name ”Ignore” and Value ”Yes”;
- The ItemDef OID “DM.BRTHDAT” creates a field OID “BRTHDAT”, with Name “BRTHDAT”, with label “Date of birth”.

See the sample ODM below:

```
<ODM FileType=”Snapshot” Granularity=”Metadata”
  CreationDateTime=”2009-05-19T14:56:39.194-00:00”
  FileOID=”PostMetaDataStudyImportDraft” ODMVersion=”1.3”
  xmlns=”http://www.cdisc.org/ns/odm/v1.3”
  xmlns:mdsol=”http://www.mdsol.com/ns/odm/metadata”>
  <Study OID=”Mediflex”>
    <GlobalVariables>
      <StudyName>Mediflex</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Mediflex</ProtocolName>
    </GlobalVariables>
    <MetaDataVersion OID=”10” Name=”2” mdsol:SignaturePrompt=”Please sign”>
    <Protocol>
      <StudyEventRef StudyEventOID=”SCREEN” Mandatory=”No” />
    </Protocol>
      <StudyEventDef OID=”SCREEN” Name=”Screen” Type=”Common”
        Repeating=”Yes”>
      <FormRef FormOID=”DM” Mandatory=”No” />
      </StudyEventDef>
      <FormDef OID=”DM” Name=”Demographic” Repeating=”No”
        mdsol:OrderNumber=”1” mdsol:SignatureRequired=”No”
        ItemGroupRef ItemGroupOID=”DM” Mandatory=”Yes” />
      </FormDef>
      <ItemGroupDef OID=”DM” Name=”DM” Repeating=”No”>
        <ItemRef ItemOID=”DM.BRTHDAT” Mandatory=”No” />
      <ItemRef ItemOID=”AE.ACTIONCD1”>
        <mdsol:Attribute Namespace=”My Integration” Name=”Ignore”
          Value=”Yes “ TransactionType=”Insert”/>
      </ItemRef>
      </ItemGroupDef>
      <ItemDef OID=”DM.BRTHDAT” Name=”BRTHDAT” DataType=”date”
        mdsol:Active=”Yes” mdsol:ControlType=”DateTime”
        <Question>
          <TranslatedText xml:lang=”en”>Date of birth</TranslatedText>
        </Question>
      </ItemDef>
    </MetaDataVersion>
  </Study>
</ODM>
```

Importing lab settings

To import lab settings for an existing draft or import lab setting while creating a new study design in Rave Architect, post ODM XML for an existing draft with lab settings to the following URL:

https://<Rave URL>/RaveWebServices/metadata/studies/<study name>/drafts

For example, for the “Mediflex” study:

https://innovate.mdsol.com/RaveWebServices/metadata/studies/Mediflex/drafts

The Rave account you use must have permissions to upload a draft.

The following sample ODM XML shows you how to import metadata into Rave to create or update a study design. The metadata elements in this example consist of these:

- The StudyName is “Mediflex”;
- The ProtocolName is “Mediflex”;
- The MetaDataVersion OID updates a draft study called “Version 1.1” in project “Mediflex”;
- The LabSettings set StandardUnits to “Conventional Unit”, with ReferenceLabs “Reference Range”, with AlertLabs “Alert Range”;
- The LabVariableMapping is LabVariable “Sex”, StudyEventOID “SCREEN”, FormOID “Demographics”, ItemOID “Sex”, RangeLocation “Earliest date”;
- The LabVariableMapping is LabVariable “Age”, StudyEventOID “SCREEN”, FormOID “Demographics”, ItemOID “Age”, RangeLocation “Earliest date”.

See the sample ODM below:

```
<ODM FileType=”Snapshot” Granularity=”Metadata”
  CreationDateTime=”2009-05-19T14:56:39.194-00:00”
  FileOID=”PostMetaDataStudyImportDraft” ODMVersion=”1.3”
  xmlns=”http://www.cdisc.org/ns/odm/v1.3”
  xmlns:mdsol=”http://www.mdsol.com/ns/odm/metadata”>
  <Study OID=”Mediflex”>
    <GlobalVariables>
      <StudyName>Mediflex</StudyName>
      <StudyDescription>
      </StudyDescription>
      <ProtocolName>Mediflex</ProtocolName>
    </GlobalVariables>
    <MetaDataVersion OID=”36” Name=”Version 1.1”>
      <mdsol:LabSettings StandardUnits=”Conventional Unit”
        ReferenceLabs=”Reference Range” AlertLabs=”Alert Range”>
      <mdsol:LabVariableMapping LabVariable=”Sex” StudyEventOID=”SCREEN”
        FormOID=”Demographics” ItemOID=”Sex” RangeLocation=”Earliest date”/>
      <mdsol:LabVariableMapping LabVariable=”Age” StudyEventOID=”SCREEN”
        FormOID=”Demographics” ItemOID=”Age” RangeLocation=”Earliest date”/>
      </mdsol:LabSettings>
```

```
    </MetaDataVersion>
  </Study>
</ODM>
```

Metadata Export Scenarios

This section contains typical examples of ODM XML metadata exported from Rave using RWSI. These examples are based on the Mediflex study which is available to registered users at https://innovate.mdsol.com, and they include:

- Listing studies
- Listing global library studies
- Listing study drafts
- Listing global library drafts
- Listing study CRF versions
- Listing global library CRF versions
- Requesting a complete CRF Version
- Requesting a complete CRF version with vendor attributes
- Requesting a complete global library CRF version with vendor attributes.

Listing studies

To obtain a list of active studies accessible by the user, request the following URL:

https://<Rave URL>/RaveWebServices/metadata/studies

For example:

https://innovate.mdsol.com/RaveWebServices/metadata/studies

The following sample ODM XML shows you a list of active studies that have the names:

- Beproxen;
- Mediflex.

See the sample ODM below:

```
<ODM ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot"
  FileOID="eb8e4688-35c4-4508-ae7a-d3cc13de9ee0"
  CreationDateTime="2009-05-19T14:56:39.194-00:00"
  xmlns="http://www.cdisc.org/ns/odm/v1.3"
  xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata">
  <Study OID="Beproxen">
    <GlobalVariables>
      <StudyName>Beproxen</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Beproxen</ProtocolName>
    </GlobalVariables>
  </Study>
  <Study OID="Mediflex">
    <GlobalVariables>
      <StudyName>Mediflex</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Mediflex</ProtocolName>
```

```
    </GlobalVariables>
  </Study>
</ODM>
```

Listing global library studies

To obtain a list of global libraries accessible by the user, request the following URL:

https://<Rave URL>/RaveWebServices/metadata/libraries

For example:

https://innovate.mdsol.com/RaveWebServices/metadata/libraries

The following ODM XML shows you a list of global libraries that have the names:

- Library One;
- Library Two.

See the sample ODM below:

```
<ODM ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot"
  FileOID="eb8e4688-35c4-4508-ae7a-d3cc13de9ee0"
  CreationDateTime="2009-05-19T14:56:39.194-00:00"
  xmlns="http://www.cdisc.org/ns/odm/v1.3"
  xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata">
  <Study OID="Library One" mdsol:ProjectType="GlobalLibraryVolume">
    <GlobalVariables>
      <StudyName>Library One</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Library One</ProtocolName>
    </GlobalVariables>
  </Study>
  <Study OID="Library Two" mdsol:ProjectType="GlobalLibraryVolume">
    <GlobalVariables>
      <StudyName>Library Two</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Library Two</ProtocolName>
    </GlobalVariables>
  </Study>
</ODM>
```

Listing study drafts

To obtain a list of study drafts accessible by the user, request the following URL:

https://<Rave URL>/RaveWebServices/metadata/studies/<study name>/drafts

For example, for the 'Mediflex' study:

https://innovate.mdsol.com/RaveWebServices/metadata/studies/Mediflex/drafts

The following ODM XML shows you the list of study drafts that have the names:

- 3.0;
- V4.0.

See the sample ODM below:

```
<ODM ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot"
  FileOID="eb8e4688-35c4-4508-ae7a-d3cc13de9ee0"
  CreationDateTime="2009-11-25T09:35:23"
  xmlns="http://www.cdisc.org/ns/odm/v1.3"
  xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata">
  <Study OID="Beproxen">
    <GlobalVariables>
      <StudyName>Beproxen</StudyName>
      <StudyDescription />
      <ProtocolName>Beproxen</ProtocolName>
    </GlobalVariables>
    <MetaDataVersion OID="23" Name="3.0" />
    <MetaDataVersion OID="6" Name="V4.0" />
  </Study>
</ODM>
```

Listing global library drafts

To obtain a list of global library drafts accessible by the user, request the following URL:

https://<Rave URL>/RaveWebServices/metadata/libraries/<global library study name>/drafts

For example, for the 'Library One' study:

https://innovate.mdsol.com/RaveWebServices/metadata/libraries/Library One/drafts

The following ODM XML shows you a list of global library drafts that have the names:

- Draft One;
- Draft Two.

See the sample ODM below:

```
<ODM ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot"
  FileOID="195c1546-cac5-411f-bfb8-13b2ff0cbfb6"
  CreationDateTime=" 2009-11-25T09:48:18 "
  xmlns="http://www.cdisc.org/ns/odm/v1.3"
  xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata">
  <Study OID="Library One" mdsol:ProjectType="GlobalLibraryVolume">
    <GlobalVariables>
      <StudyName>Library One</StudyName>
      <StudyDescription/>
      <ProtocolName>Library One</ProtocolName>
    </GlobalVariables>
    <MetaDataVersion OID="44" Name="Draft One"/>
```

```
    <MetaDataVersion OID="12" Name="Draft Two"/>
  </Study>
</ODM>
```

Listing study CRF versions

To obtain a list of active study CRF versions accessible by the user, request the following URL:

https://<Rave URL>/RaveWebServices/metadata/studies/<study name>/versions

For example, for the 'Mediflex' study:

https://innovate.mdsol.com/RaveWebServices/metadata/studies/Mediflex/versions

The following sample ODM XML shows you a list of active study CRF versions that have the names:

- V2.1;
- V2.0;
- V1.0.

See the sample ODM below:

```
<ODM ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot"
  FileOID="195c1546-cac5-411f-bfb8-13b2ff0cbfb6"
  CreationDateTime=" 2009-11-25T09:48:18 "
  xmlns="http://www.cdisc.org/ns/odm/v1.3"
  xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata">
 <Study OID="Mediflex">
    <GlobalVariables>
      <StudyName>Mediflex</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Mediflex</ProtocolName>
    </GlobalVariables>
    <MetaDataVersion OID="27" Name="V2.1" />
    <MetaDataVersion OID="6" Name="V2.0" />
    <MetaDataVersion OID="4" Name="V1.0" />
  </Study>
</ODM>
```

Listing global library CRF versions

To obtain a list of active global library CRF versions accessible by the user, request the following URL:

https://<Rave URL>/RaveWebServices/metadata/libraries/<global library study name>/versions

For example, for the 'Library One' library:

https://innovate.mdsol.com/RaveWebServices/metadata/libraries/Library One/versions

The following sample ODM XML shows you a list of global library CRF versions that have the names:

- 2;
- 1.

See the sample ODM below:

```
<ODM ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot"
  FileOID="195c1546-cac5-411f-bfb8-13b2ff0cbfb6"
  CreationDateTime=" 2009-11-25T09:48:18 "
  xmlns="http://www.cdisc.org/ns/odm/v1.3"
  xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata">
  <Study OID="Library One" mdsol:ProjectType="GlobalLibraryVolume">
    <GlobalVariables>
        <StudyName>Library One</StudyName>
        <StudyDescription></StudyDescription>
        <ProtocolName>Library One</ProtocolName>
    </GlobalVariables>
    <MetaDataVersion OID="11" Name="2" />
    <MetaDataVersion OID="8" Name="1" />
  </Study>
</ODM>
```

Requesting a complete study CRF version

To obtain a complete study CRF version accessible by the user, request the following URL:

https://<Rave URL>/RaveWebServices/metadata/studies/<study name>/versions/<version number>

For example, for the 'Mediflex' study:

https://innovate.mdsol.com/RaveWebServices/metadata/studies/Mediflex/versions/209

Note: Vendor extension

Metadata containing lab settings will return the values using mdsol vendor extensions.

The following sample ODM XML shows you a complete study CRF. The metadata elements in this example describe:

- A study named "Mediflex";
- A CRF version named "1";
- A subject-level form with OID "DM" named "Demographic";
- A field with OID "RACE";
- And a data dictionary with OID "RACE" named "Race".

See the sample ODM below:

```
<ODM ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot"
  FileOID="195c1546-cac5-411f-bfb8-13b2ff0cbfb6"
  CreationDateTime=" 2009-11-25T09:48:18 "
```

```
  xmlns="http://www.cdisc.org/ns/odm/v1.3"
  xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata">
  <Study OID="Mediflex">
    <GlobalVariables>
      <StudyName>Mediflex</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Mediflex</ProtocolName>
    </GlobalVariables>
    <BasicDefinitions></BasicDefinitions>
    <MetaDataVersion OID="209" Name="1">
      <Protocol>
        <StudyEventRef StudyEventOID="SUBJECT" Mandatory="Yes" />
      </Protocol>
      <StudyEventDef OID="SUBJECT" Name="Subject" Type="Common"
        Repeating="No">
        <FormRef FormOID="DM" OrderNumber="1" Mandatory="No" />
      </StudyEventDef>
      <FormDef OID="DM" Name="Demographic" Repeating="Yes">
        <ItemGroupRef ItemGroupOID="DM" Mandatory="Yes" />
      </FormDef>
      <ItemGroupDef OID="DM" Name="DM" Repeating="No">
        <ItemRef ItemOID="DM.RACE" OrderNumber="1" Mandatory="No" />
      </ItemGroupDef>
      <ItemDef OID="DM.RACE" Name="RACE" DataType="integer" Length="1"
        mdsol:ControlType="DropDownList">
      </ItemDef>
      <CodeList OID="RACE" Name="Race" DataType="integer">
        <CodeListItem CodedValue="1" mdsol:OrderNumber="1">
          <Decode>
            <TranslatedText xml:lang="en">Asian</TranslatedText>
            <TranslatedText xml:lang="ja">Asian</TranslatedText>
          </Decode>
         </CodeListItem>
      </CodeList>
    </MetaDataVersion>
  </Study>
</ODM>
```

Requesting a complete global library CRF version

To obtain a complete global library CRF version accessible by the user, request the following URL:

https://<Rave URL>/RaveWebServices/metadata/libraries/<global library study name>/versions/<version number>

For example, for the 'Library One' library:
https://innovate.mdsol.com/RaveWebServices/metadata/libraries/Library One/versions/8

The following sample ODM XML shows you how a complete global library CRF version. The metadata elements in this example describe:

- A library named "Library One";
- A CRF version named "1";
- A subject-level form with OID "DM" named "Demographic";
- A field with OID "RACE";
- And a data dictionary with OID "RACE" named "Race".

See the sample ODM below:

```
<ODM ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot"
  FileOID="195c1546-cac5-411f-bfb8-13b2ff0cbfb6"
  CreationDateTime=" 2009-11-25T09:48:18 "
  xmlns="http://www.cdisc.org/ns/odm/v1.3"
  xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata">
  <Study OID="Library One" mdsol:ProjectType="GlobalLibraryVolume">
    <GlobalVariables>
      <StudyName>Library One</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Library One</ProtocolName>
    </GlobalVariables>
    <BasicDefinitions></BasicDefinitions>
    <MetaDataVersion OID="8" Name="1">
      <Protocol>
        <StudyEventRef StudyEventOID="SUBJECT" Mandatory="Yes" />
      </Protocol>
      <StudyEventDef OID="SUBJECT" Name="Subject" Type="Common"
        Repeating="No">
        <FormRef FormOID="DM" OrderNumber="1" Mandatory="No" />
      </StudyEventDef>
      <FormDef OID="DM" Name="Demographic" Repeating="Yes">
        <ItemGroupRef ItemGroupOID="DM" Mandatory="Yes" />
      </FormDef>
      <ItemGroupDef OID="DM" Name="DM" Repeating="No">
        <ItemRef ItemOID="DM.RACE" OrderNumber="1" Mandatory="No" />
      </ItemGroupDef>
```

```
      <ItemDef OID="DM.RACE" Name="RACE" DataType="integer" Length="1"
        mdsol:ControlType="DropDownList">
      </ItemDef>
      <CodeList OID="RACE" Name="Race" DataType="integer">
        <CodeListItem CodedValue="1" mdsol:OrderNumber="1">
          <Decode>
            <TranslatedText xml:lang="en">Asian</TranslatedText>
          </Decode>
         </CodeListItem>
      </CodeList>
    </MetaDataVersion>
  </Study>
</ODM>
```

Requesting a complete CRF version with vendor attributes

To obtain a complete CRF version with vendor attributes accessible by the user, request the following URL:

https://<Rave URL>/RaveWebServices/metadata/studies/<study name>/versions/<version number>/attributes?namespace=<client reference>

For example, to read the 'MyIntegration' attributes for CRF version '209' of the 'Mediflex' study:

https://innovate.mdsol.com/RaveWebServices/metadata/studies/Mediflex/versions/209/attributes?namespace=MyIntegration

Note: Attribute with Different Namespace

A field can store more than one attribute with different namespace but listing would be restricted to the namespace in the URL.

The following sample ODM XML shows you a complete CRF version with vendor attributes. The metadata elements in this example describe:

- A study named "Mediflex";
- A CRF version named "1";
- A subject-level form with OID "DM" named "Demographic";
- A field with OID "RACE";
- And an attribute with name "Ignore" and value "Yes" in the namespace "MyIntegration".

See the sample ODM below:

```
<ODM ODMVersion="1.3" Granularity="Metadata" FileType="Snapshot"
  FileOID="195c1546-cac5-411f-bfb8-13b2ff0cbfb6"
  CreationDateTime=" 2009-11-25T09:48:18 "
```

```
  xmlns="http://www.cdisc.org/ns/odm/v1.3"
  xmlns:mdsol="http://www.mdsol.com/ns/odm/metadata">
  <Study OID="Mediflex">
    <GlobalVariables>
      <StudyName>Mediflex</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Mediflex</ProtocolName>
    </GlobalVariables>
    <BasicDefinitions></BasicDefinitions>
    <MetaDataVersion OID="209" Name="1">
      <Protocol>
        <StudyEventRef StudyEventOID="SUBJECT" Mandatory="Yes" />
      </Protocol>
      <StudyEventDef OID="SUBJECT" Name="Subject" Type="Common"
        Repeating="No">
        <FormRef FormOID="DM" OrderNumber="1" Mandatory="No" />
      </StudyEventDef>
      <FormDef OID="DM" Name="Demographic" Repeating="Yes">
        <ItemGroupRef ItemGroupOID="DM" Mandatory="Yes" />
      </FormDef>
      <ItemGroupDef OID="DM" Name="DM" Repeating="No">
        <ItemRef ItemOID="DM.RACE" OrderNumber="1" Mandatory="No">
          <mdsol:Attribute Namespace="MyIntegration" Name="Ignore"
            Value="Yes"/>
      </ItemGroupDef>
      <ItemDef OID="DM.RACE" Name="RACE" DataType="integer" Length="1"
        mdsol:ControlType="DropDownList">
      </ItemDef>
    </MetaDataVersion>
  </Study>
</ODM>
```

Requesting a complete global library CRF version with vendor attributes

To obtain a complete global library CRF version with vendor attributes accessible by the user, request the following URL:

https://<Rave URL>/RaveWebServices/metadata/libraries/<global library study name>/versions/<version number>/attributes?namespace=<client reference>

For example, to read the 'MyIntegration' attributes for CRF version '209' of the 'Library One' study:

https://innovate.mdsol.com/RaveWebServices/metadata/libraries/Library One/versions/209/attributes?namespace=MyIntegration

The following ODM XML shows you a complete global library CRF version with vendor attributes. The metadata elements in this example describe:

- A library named “Library One”;
- A CRF version named “1”;
- A subject-level form with OID “DM” named “Demographic”;
- A field with OID “RACE”;
- And an attribute with name “Ignore” and value “Yes” in the namespace “MyIntegration”.

See the sample ODM below:

```
<ODM ODMVersion=”1.3” Granularity=”Metadata” FileType=”Snapshot”
  FileOID=”195c1546-cac5-411f-bfb8-13b2ff0cbfb6”
  CreationDateTime=” 2009-11-25T09:48:18 “
  xmlns=”http://www.cdisc.org/ns/odm/v1.3”
  xmlns:mdsol=”http://www.mdsol.com/ns/odm/metadata”>
  <Study OID=”Library One” mdsol:ProjectType=”GlobalLibraryVolume”>
    <GlobalVariables>
      <StudyName>Library One</StudyName>
      <StudyDescription></StudyDescription>
      <ProtocolName>Library One</ProtocolName>
    </GlobalVariables>
    <BasicDefinitions></BasicDefinitions>
    <MetaDataVersion OID=”209” Name=”1”>
      <Protocol>
        <StudyEventRef StudyEventOID=”SUBJECT” Mandatory=”Yes” />
      </Protocol>
      <StudyEventDef OID=”SUBJECT” Name=”Subject” Type=”Common”
        Repeating=”No”>
        <FormRef FormOID=”DM” OrderNumber=”1” Mandatory=”No” />
      </StudyEventDef>
      <FormDef OID=”DM” Name=”Demographic” Repeating=”Yes”>
        <ItemGroupRef ItemGroupOID=”DM” Mandatory=”Yes” />
      </FormDef>
      <ItemGroupDef OID=”DM” Name=”DM” Repeating=”No”>
        <ItemRef ItemOID=”DM.RACE” OrderNumber=”1” Mandatory=”No”>
          <mdsol:Attribute Namespace=”MyIntegration” Name=”Ignore”
            Value=”Yes”/>
      </ItemGroupDef>
      <ItemDef OID=”DM.RACE” Name=”RACE” DataType=”integer” Length=”1”
        mdsol:ControlType=”DropDownList”>
      </ItemDef>
    </MetaDataVersion>
  </Study>
```

```
</ODM>
```

Clinical Data Import Scenarios

This section contains typical examples of ODM XML clinical data for import into Rave using RWSI and shows the results of these imports in Rave. These examples are based on the Mediflex study which is available to registered users at https://innovate.mdsol.com. Log in using the details provided by Medidata.

- Enrolling a Subject (by Inserting a Subject and Updating a Form)
- Randomizing a Subject (by Inserting a Form)
- Adding an Unscheduled Visit (by Inserting a Folder and a Form)
- Recording Vital Signs (by Updating a Form in a Nested Folder)
- Recording Adverse Events (by Inserting Log Lines)
- Recording ePRO Data (by Inserting and Removing Log Lines across Multiple Forms)

Enrolling a Subject (by Inserting a Subject and Updating a Form)

The following ODM XML request examples show a subject being added and subsequently updated. There are two separate transactions.

To enrol a subject, the subject must be added in Rave; this is shown in the example below:

```
<ODM xmlns='http://www.cdisc.org/ns/odm/v1.3' ODMVersion='1.3'
FileType='Transactional' FileOID='Example-1' CreationDateTime='2008-01-
01T00:00:00'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='123 ABC' TransactionType='Insert'>
      <SiteRef LocationOID='4567'/>
    </SubjectData>
  </ClinicalData>
</ODM>
```

Notes:

1. The Rave project name and environment for this study is specified in the StudyOID element.
2. The MetaDataVersionOID is ignored by RWSI, but **must** be present for this to be a valid ODM.
3. The TransactionType of 'Insert' indicates that we are adding this subject to Rave.
4. The SiteRef is mandatory as Rave needs to know which site the new subject will belong to. The site must already exist in Rave before the request to add a subject is submitted.

See the 'Rave Web Services Inbound v1.0 API Quick Reference' guide for more information on how to correctly construct the request ODM XML.

After this request XML has been successfully imported, the following will be visible in Rave:

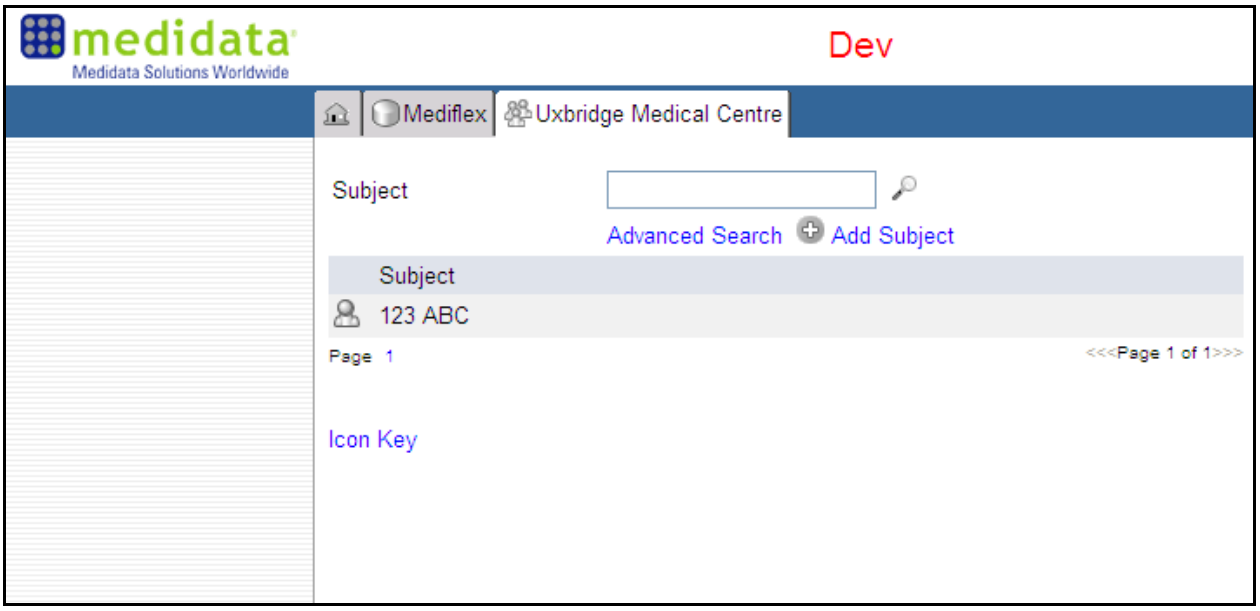

Figure 1: The enrolled subject in the Mediflex(Dev) study in Rave

Now, the demographic information needs to be updated for the subject. To do this, a second ODM XML request is required:

```
<ODM xmlns='http://www.cdisc.org/ns/odm/v1.3' ODMVersion='1.3'
FileType='Transactional' FileOID='Example-2' CreationDateTime='2008-01-
01T00:00:00'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='123 ABC' TransactionType='Update'>
      <SiteRef LocationOID='4567'/>
      <StudyEventData StudyEventOID='SCREEN'>
        <FormData FormOID='DM'>
          <ItemGroupData ItemGroupOID='DM'>
            <ItemData ItemOID='BRTHDTC' Value='01 JAN 1980'/>
            <ItemData ItemOID='SEX' Value='MALE'/>
            <ItemData ItemOID='RACE' Value='3'/>
            <ItemData ItemOID='COUNTRY' Value='GBR'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>
```

Notes:

1. The TransactionType for the subject is 'Update' because the subject already exists.
2. All corresponding folders, forms and fields already exist in Rave as they are

created when the subject is enrolled.

3. Although not specified, the TransactionType for elements beneath the SubjectData element is also 'Update' because this is inherited from the SubjectData element. The TransactionType can be repeated on child elements if desired, and should be specified if the child requires a different TransactionType.

4. The StudyEventData/StudyEventOID 'SCREEN' corresponds to the Rave 'Screening' folder with OID 'SCREEN'.

5. The FormData/FormDataOID 'DM' corresponds to the Rave 'Demographics' form with OID 'DM'.

6. Each of the ItemData elements correspond to the Rave fields with the given OIDs.

7. The 'SEX', 'RACE' and 'COUNTRY' ItemData/ItemOID values provide the coded values for the associated data dictionaries in Rave.

8. The site reference is optional and can be left out for 'update' and 'remove' TransactionTypes if the subject's name is unique across the whole study.

9. Any date values **must** be in the corresponding Rave field's date format, e.g. 'dd MMM YYYY'. See the BRTHDTC item value in the example.

See the 'Rave Web Services Inbound v1.0 API Quick Reference' guide for more information on how to correctly construct the request ODM XML.

After this request XML has been successfully imported, the following will be visible in Rave:

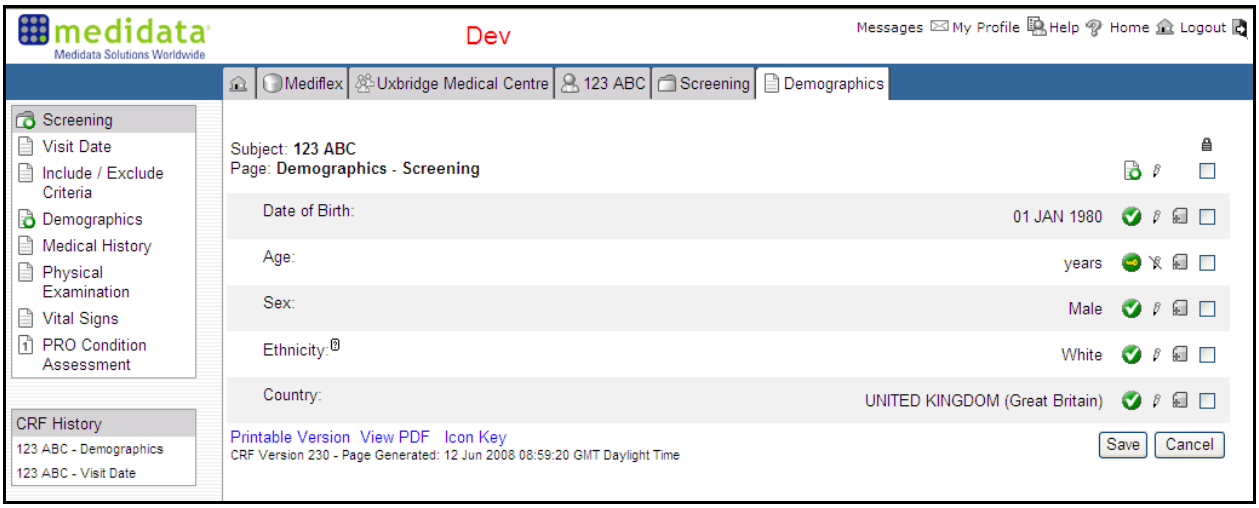

Figure 2: The demographic data for the enrolled subject

Note:

Notice in Figure 2 that the Age field in the demographics page is not populated. This is because derived fields cannot be updated through RWSI. If RWSI receives clinical data for a derived datapoint, it will return an error 'Transaction on a derived field is not permitted' (RWS00049). The Age field will be calculated by Rave when the Visit Date field on the Visit Date form is populated.

Randomizing a Subject (by Inserting a Form)

To randomize a subject in this example study, firstly the Randomization form is added to the Screening folder and then the Date of Randomization field value is set. The following ODM XML request can be sent to achieve this:

```
<ODM xmlns='http://www.cdisc.org/ns/odm/v1.3' ODMVersion='1.3'
FileType='Transactional' FileOID='Example-3' CreationDateTime='2008-01-
01T00:00:00'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='123 ABC' TransactionType='Update'>
      <SiteRef LocationOID='4567'/>
      <StudyEventData StudyEventOID='SCREEN'>
        <FormData FormOID='RAND' TransactionType='Insert'>
          <ItemGroupData ItemGroupOID='RAND'>
            <ItemData ItemOID='DSSTDT' Value='01 JAN 2008'/>
            <ItemData ItemOID='ARMCD' Value='2'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>
```

Notes:

1. The TransactionType for the form is 'Insert' to indicate that the form is being added to Rave.
2. The TransactionType of 'Insert' is inherited by all elements underneath the form as these are also being added.
3. To update the Date of Randomization in a later transaction, the TransactionType on the form would need to become 'Update'. Alternatively, the TransactionType element on the form could be removed entirely and the value inherited from the subject.

See the 'Rave Web Services Inbound v1.0 API Quick Reference' guide for more information on how to correctly construct the request ODM XML.

After this request XML has been successfully imported, the following will be visible in Rave:

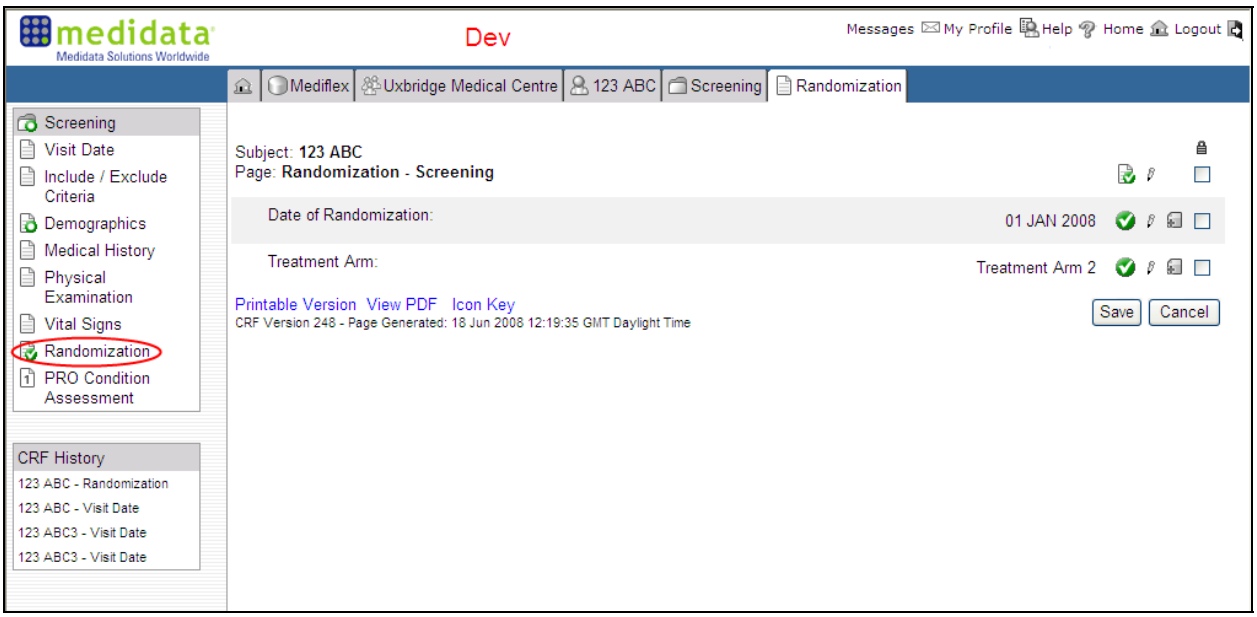

Figure 3: The randomized subject

Note:
In Figure 3, notice the new Randomization menu entry in the sidebar (circled).

Adding an Unscheduled Visit (by Inserting a Folder and a Form)

To add an unscheduled visit in this example study it is necessary to add a new unscheduled visit folder and visit date form in Rave. The following ODM XML request can be sent to achieve this:

```
<ODM xmlns='http://www.cdisc.org/ns/odm/v1.3' ODMVersion='1.3'
FileType='Transactional' FileOID='Example-4' CreationDateTime='2008-01-
01T00:00:00'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='123 ABC' TransactionType='Update'>
      <SiteRef LocationOID='4567'/>
      <StudyEventData StudyEventOID='UNSCHEDULED' StudyEventRepeatKey='1'
TransactionType='Insert'>
        <FormData FormOID='VISIT'>
          <ItemGroupData ItemGroupOID='VISIT'>
            <ItemData ItemOID='DTC' Value='21 FEB 2008'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
</ClinicalData>
```

```
</ODM>
```

Notes:

1. As the example above contains a repeating folder, the ItemGroupRepeatKey is specified.

2. The visit date triggers an edit check in Rave that renames the instance of the folder from 'Visit (1)' to 'Visit 21 Feb 2008'.

3. To update the visit date, change TransactionType from 'Insert' to 'Update'.

4. For each new unscheduled visit, the StudyEventRepeatKey would increment by one.

5. StudyEventData elements correspond to a folder in Rave and StudyEventData elements that have a StudyEventRepeatKey show the position of this folder in relation to other folders with the same OID in Rave. The StudyEventRepeatKey must start at one and increment by one sequentially without any gaps. For example, if another StudyEventData element were added which had a StudyEventRepeatKey, it **must** be '2' and not any other number. RWSI will return the error 'Web Service currently unavailable'. This incorrect error message reported in this situation is a known issue and will be fixed in a subsequent release of the software

See the 'Rave Web Services Inbound v1.0 API Quick Reference' guide for more information on how to correctly construct the request ODM XML.

After this request XML has been successfully imported, the following will be visible in Rave:

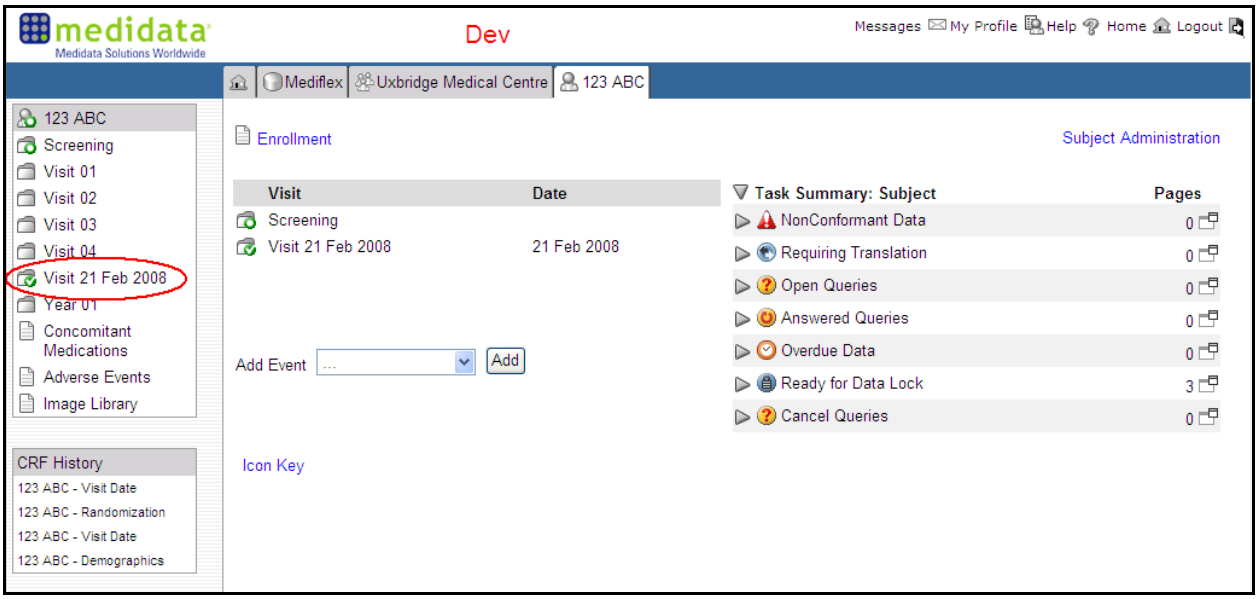

Figure 4: The unscheduled visit

Note:
In Figure 4, notice the new 'Visit 21 Feb 2008' menu entry in the sidebar for the new unscheduled visit (circled in red).

The 'Visit 21 Feb 2008' page shows the following information:

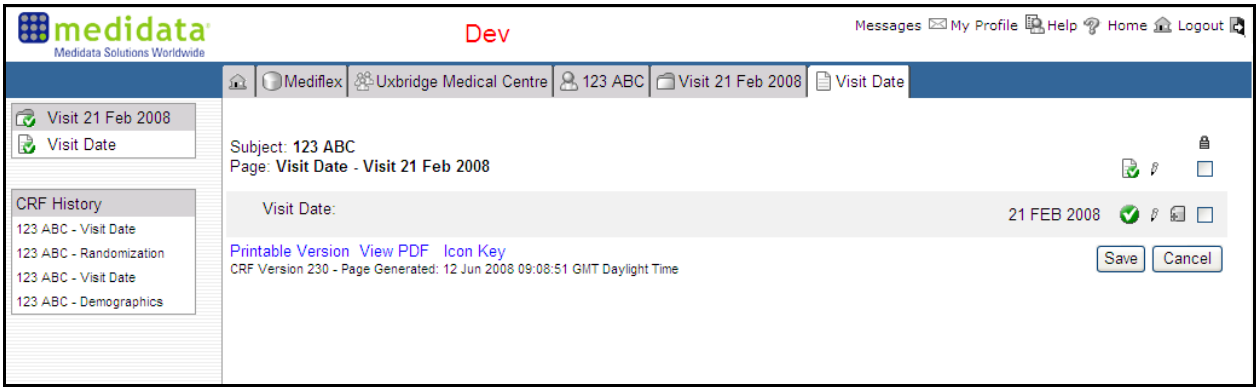

Figure 5: The Visit 21 Feb 2008 page

Recording Vital Signs (by Updating a Form in a Nested Folder)

To record vital signs in this example study we need to update the Vital Signs form which is in the Cycle 01 folder, which is nested in the Year 01 folder. The following ODM XML request can be sent to achieve this:

```
<ODM xmlns='http://www.cdisc.org/ns/odm/v1.3' ODMVersion='1.3'
FileType='Transactional' FileOID='Example-4' CreationDateTime='2008-01-
01T00:00:00'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='123 ABC' TransactionType='Update'>
      <SiteRef LocationOID='4567'/>
      <StudyEventData StudyEventOID='YEAR1'
StudyEventRepeatKey='YEAR1[1]/CYCLE1[1]'>
        <FormData FormOID='VS'>
          <ItemGroupData ItemGroupOID='VS'>
            <ItemData ItemOID='VSSTAT' Value='Y'/>
            <ItemData ItemOID='VSDTC' Value='01 JAN 2008'/>
            <ItemData ItemOID='SYSBP' Value='120'/>
            <ItemData ItemOID='DIABP' Value='80'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>
```

Note:
The folder location of the form is specified using the StudyEventRepeatKey attribute.

See 'Appendix A' in the 'Rave Web Services Inbound v1.0 API Quick Reference' guide for more information on how to use the StudyEventRepeatKey to specify folder locations and how to correctly construct the request ODM XML.

After this request XML has been successfully imported, the following will be visible in Rave:

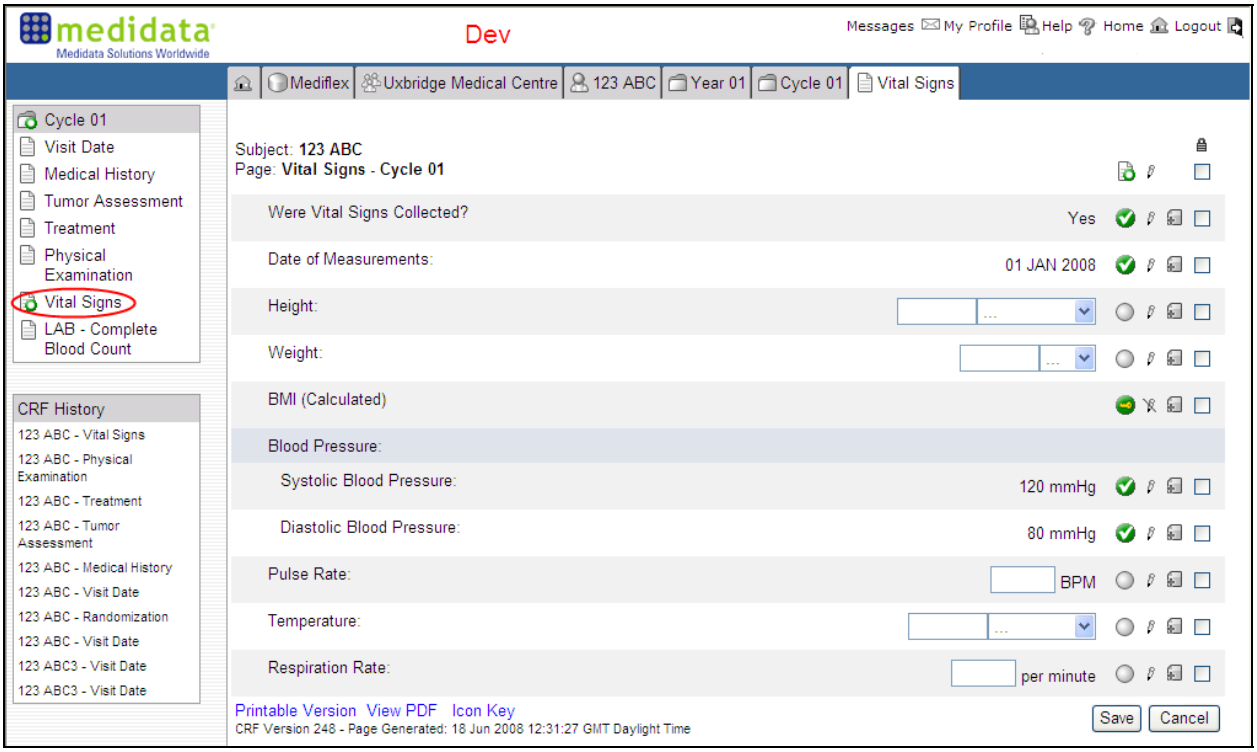

Figure 6: The Vital Signs page

Note:
In Figure 6, notice the new Vital Signs menu entry in the sidebar (circled).

Recording Adverse Events (by Inserting Log Lines)

To record adverse events in this example study, the data in two log lines and one normal field is added to the Adverse Events form which is held at the subject level in Rave. The following ODM XML request can be sent to achieve this:

```
ODM xmlns='http://www.cdisc.org/ns/odm/v1.3' ODMVersion='1.3'
FileType='Transactional' FileOID='Example-6' CreationDateTime='2008-01-
01T00:00:00'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='123 ABC' TransactionType='Update'>
      <SiteRef LocationOID='4567'/>
      <StudyEventData StudyEventOID='SUBJECT'>
        <FormData FormOID='AE'>
          <ItemGroupData ItemGroupOID='AE'>
            <ItemData ItemOID='AEYN' Value='Y'/>
          </ItemGroupData>
          <ItemGroupData ItemGroupOID='AE_LOG_LINE' ItemGroupRepeatKey='1'>
            <ItemData ItemOID='AETERM' Value='Headache'/>
            <ItemData ItemOID='AESTDTC' Value='01 JAN 2008'/>
            <ItemData ItemOID='AEONG' Value='N'/>
            <ItemData ItemOID='AEENDTC' Value='01 JAN 2008'/>
          </ItemGroupData>
          <ItemGroupData ItemGroupOID='AE_LOG_LINE' ItemGroupRepeatKey='2'
TransactionType='Insert'>
            <ItemData ItemOID='AETERM' Value='Joint pain'/>
            <ItemData ItemOID='AESTDTC' Value='01 JAN 2008'/>
            <ItemData ItemOID='AEONG' Value='Y'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>
```

Notes:

1. The AE Form is present in the study at the subject level, so the special study event OID of 'SUBJECT' is used to indicate this.

2. This Adverse Event form was created automatically by Rave when the subject was created (via the default matrix). This was done in the 'Enrolling a Subject' section. During this process, Rave creates an empty first log line, which corresponds to the item group with repeat key '1'. Therefore, the transaction type for this item group must be 'Update'. Subsequent item groups do not have a corresponding log line in Rave and so the transaction type must be 'Insert'.

3. The items that correspond to log line fields in Rave occur within item groups that have an ItemGroupRepeatKey. This ItemGroupRepeatKey is the log line number in Rave. It must start at one and increment by one sequentially without any gaps. For example, if another item group were added, it **must** be '3' and not any other number.

4. It is not necessary to set the value of all fields present in a form or log line in Rave in a single ODM XML request.

5. To update values in an existing log line, the TransactionType would need to

be changed to 'Update'.

See the 'Rave Web Services Inbound v1.0 API Quick Reference' guide for more information on how to correctly construct the request ODM XML.

After this request XML has been successfully imported, the following will be visible in Rave:

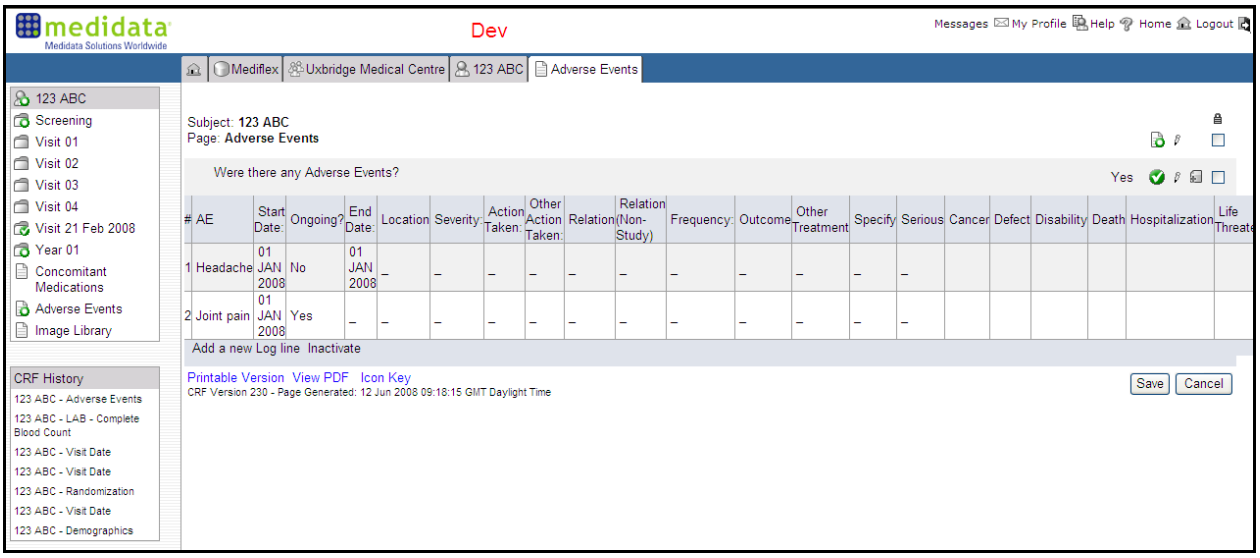

Figure 7: The adverse events for the subject

Recording ePRO Data (by Inserting and Removing Log Lines across Multiple Forms)

To record ePRO data in this example study, the Pain Diary form will be updated. This form is repeated in each visit, so the data is placed into the corresponding study events. The following ODM XML request can be sent to achieve this:

```
<ODM xmlns='http://www.cdisc.org/ns/odm/v1.3' ODMVersion='1.3'
FileType='Transactional' FileOID='Example-7' CreationDateTime='2008-01-
01T00:00:00'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='123 ABC' TransactionType='Update'>
      <SiteRef LocationOID='4567'/>
      <StudyEventData StudyEventOID='VISIT01'>
        <FormData FormOID='FORM_PAIN_SI'>
          <ItemGroupData ItemGroupOID='FORM_PAIN_SI_LOG_LINE'
ItemGroupRepeatKey='1'>
            <ItemData ItemOID='IT_DATE' Value='2008 01 01'/>
            <ItemData ItemOID='IT_TIME' Value='12:00:00'/>
            <ItemData ItemOID='IT_SEVERE' Value='50'/>
            <ItemData ItemOID='IT_REC_ID' Value='12345678'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
      <StudyEventData StudyEventOID='VISIT02'>
```

```
        <FormData FormOID='FORM_PAIN_SI'>
          <ItemGroupData ItemGroupOID='FORM_PAIN_SI_LOG_LINE'
ItemGroupRepeatKey='1'>
            <ItemData ItemOID='IT_DATE' Value='2008 02 01'/>
            <ItemData ItemOID='IT_TIME' Value='18:00:00'/>
            <ItemData ItemOID='IT_SEVERE' Value='75'/>
            <ItemData ItemOID='IT_REC_ID' Value='12345679'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>
```

Note:

The TransactionType is 'Update' throughout because the corresponding forms and log lines already exist.

See the 'Rave Web Services Inbound v1.0 API Quick Reference' guide for more information on how to correctly construct the request ODM XML.

After this request XML has been successfully imported, the following visit folders will be visible in Rave:

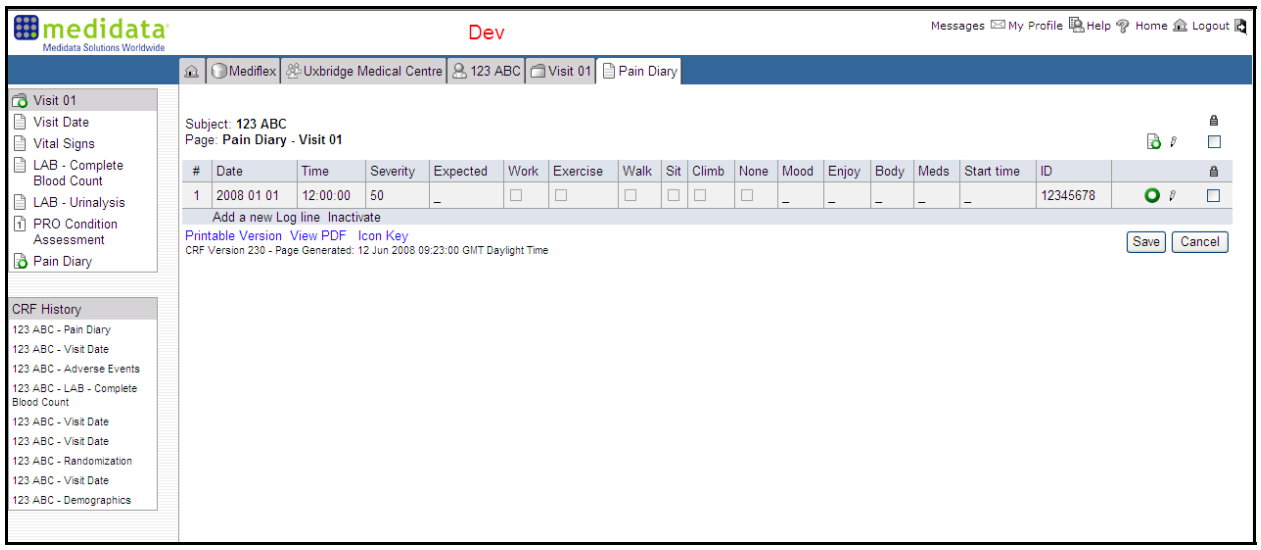

Figure 8: The Visit01 folder

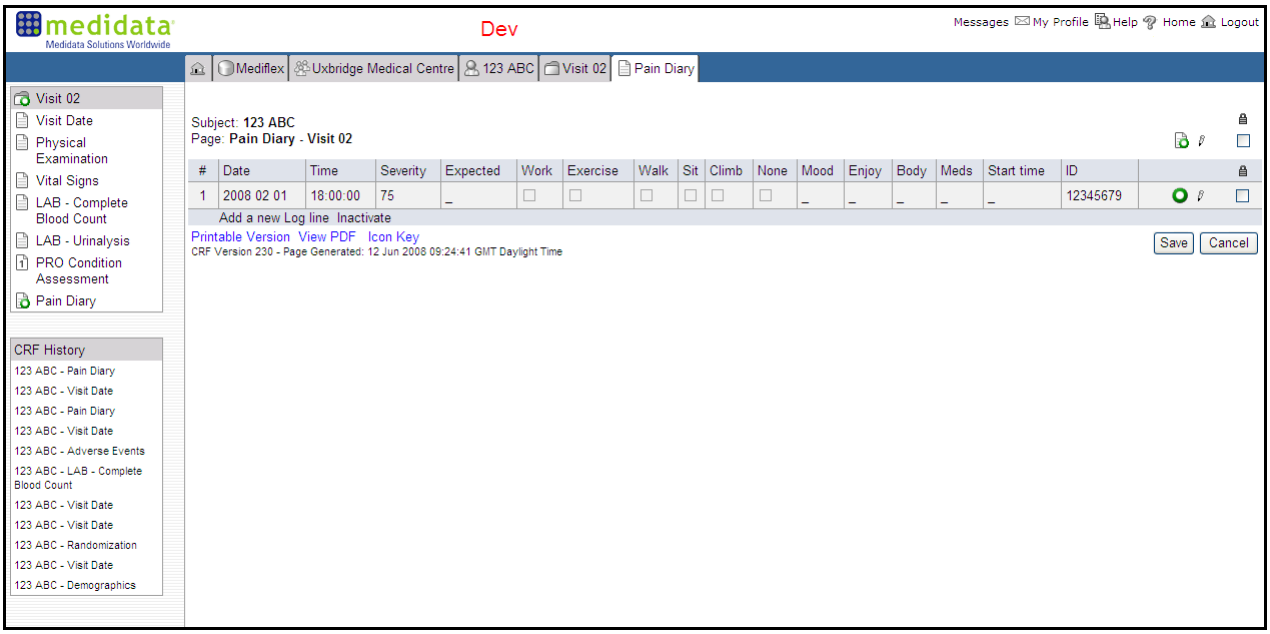

Figure 9: The Visit02 folder

To inactivate an incorrectly added record, a transaction using the 'Remove' transaction type on the item group can be sent. This example also adds a second log line.

```
<ODM xmlns='http://www.cdisc.org/ns/odm/v1.3' ODMVersion='1.3'
FileType='Transactional' FileOID='Example-8' CreationDateTime='2008-01-
01T00:00:00'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='123 ABC' TransactionType='Update'>
      <SiteRef LocationOID='4567'/>
      <StudyEventData StudyEventOID='VISIT01'>
        <FormData FormOID='FORM_PAIN_SI'>
          <ItemGroupData ItemGroupOID='FORM_PAIN_SI_LOG_LINE'
ItemGroupRepeatKey='1' TransactionType='Remove'/>
          <ItemGroupData ItemGroupOID='FORM_PAIN_SI_LOG_LINE'
ItemGroupRepeatKey='2' TransactionType='Insert'>
            <ItemData ItemOID='IT_DATE' Value='2008 01 02'/>
            <ItemData ItemOID='IT_TIME' Value='18:00:00'/>
            <ItemData ItemOID='IT_SEVERE' Value='50'/>
            <ItemData ItemOID='IT_REC_ID' Value='12345680'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>
```

Note:

The additional item group with repeat key '2' must have a transaction type of 'Insert'.

See the 'Rave Web Services Inbound v1.0 API Quick Reference' guide for more information on how to correctly construct the request ODM XML.

After this request XML has been successfully imported, Rave will show the inactivated record (shown crossed-out in Rave) and the added one.

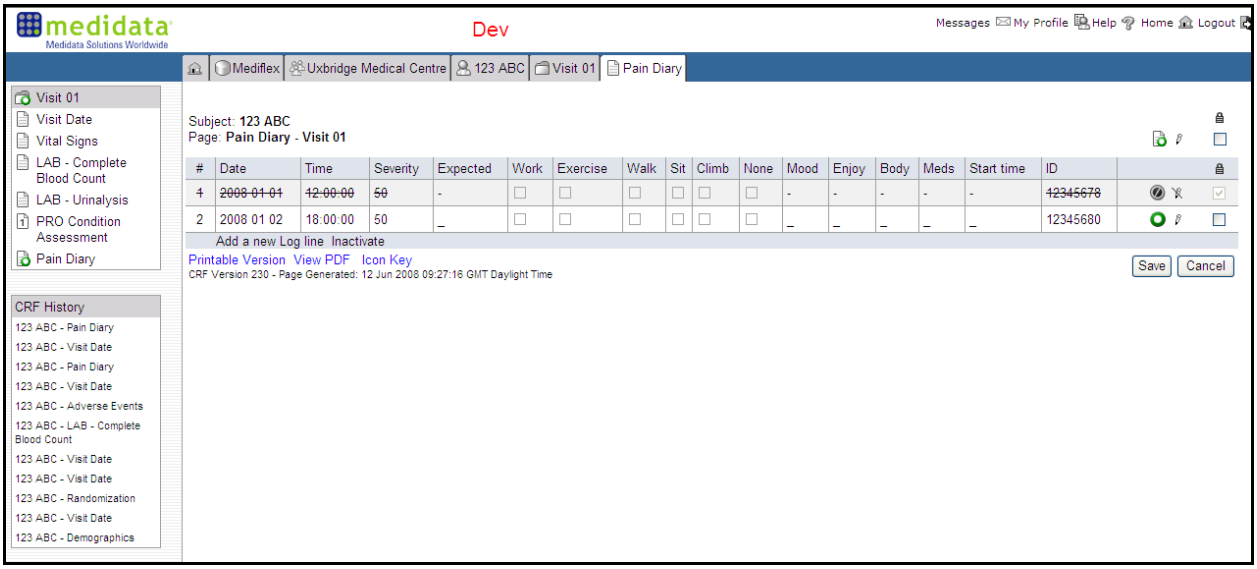

Figure 10: The inactivated record and the new record

Notes:

1. Subjects cannot be removed from Rave. If RWSI receives a request to remove a subject, it will return an error 'Removing a subject is not permitted' in its response (RWS00054).

2. For all other elements apart from Subject, a 'Remove' transaction will inactivate their applicable equivalents in Rave. Their data will remain in the Rave database, but will be marked as inactive (as shown crossed-out in Figure 10). This means that they can be reactivated by sending a request containing the appropriate elements and values with a TransactionType of 'Insert'.

Notes:

1. If a deactivated folder which contains a form is reactivated, only the folder will be reactivated. The form will remain deactivated. Reactivating the form will not be possible unless fields are inserted in the form.

2. If a user tries to insert a log line record in a form that does not have log line records, RWSI will return the error 'Non log line form, cannot insert multiple

records' (RWS00101).

Notes:

1. The user cannot insert to a restricted log line.

2. If a user tries to insert to a restricted log line, RWSI will return the error 'Record restricted by max limit' (RWS00088).

Notes:

1. The user cannot delete from a restricted log line.

2. If a user tries to delete from a restricted log line, RWSI will return the error 'Record inactivation not authorized' (RWS00089).

Notes:

1. The user cannot use dynamic search list.

2. If a user tries to use dynamic search list, RWSI will return the error 'Dynamic search list not supported' (RWS00095).

Appendix A

RWSI response code description table

The following table provides a list of all the RWSI response codes and descriptions returned following an import transaction.

RWSI Response code	HTTP return code	Client exception / error message
N/A	HTTP Code 200 This signifies a successful import	N/A
RWS00001	HTTP Code 500 (Internal Server error)	Internal Error – Web Service currently unavailable
RWS00002	HTTP Code 500 (Internal Server error)	Internal Error – Web Service currently unavailable
RWS00004	HTTP Code 500 (Internal Server error)	Internal Error – Web Service currently unavailable
RWS00005	HTTP code 500 (Internal Server error)	Internal Error – Web Service currently unavailable
RWS00006	HTTP Code 500 (Internal Server error)	Unexpected internal error
RWS00007	HTTP code 503 (service unavailable)	This Rave web service is under maintenance
RWS00008	HTTP code 401 (unauthorised)	Incorrect login and password combination
RWS00009	HTTP code 403 (forbidden)	Insecure HTTP requests are forbidden by configuration. Only secure HTTPS requests are allowed
RWS00010	HTTP code 400 (bad request)	This is not a valid XML document
RWS00011	HTTP code 400 (bad request)	This is not a valid ODM 1.2 standard document
RWS00012	HTTP code 400 (bad request)	Filetype <Filetype Value> not supported
RWS00013	HTTP code 400 (bad request)	Data not imported as more than one study was found in the document
RWS00014	HTTP code 404 (not found)	Study does not exist

RWSI Response code	HTTP return code	Client exception / error message
RWS00015	HTTP code 409 (conflict)	Site has no CRF version assigned
RWS00016	HTTP code 409 (conflict)	Site not active
RWS00017	HTTP code 404 (not found)	Site does not exist
RWS00018	HTTP code 404 (not found)	Subject not authorised
RWS00019	HTTP code 409 (conflict)	Subject not active
RWS00020	HTTP code 400 (bad request)	Data for more than one subject was found in the document
RWS00021	HTTP code 409 (conflict)	Transaction not processed as more than one subject has the same identifier at the given site
RWS00022	HTTP code 409 (conflict)	Transaction not processed as more then one subject (without site details) identified in study
RWS00023	HTTP code 404 (not found)	Subject does not exist
RWS00024	HTTP code 409 (conflict)	Subject already exists
RWS00025	HTTP code 400 (bad request)	Subjects must have site details to be created
RWS00026	HTTP code 404 (not found)	Folder not authorised
RWS00027	HTTP code 409 (conflict)	Folder does not exist
RWS00028	HTTP code 404 (not found)	Folder not found
RWS00029	HTTP code 409 (conflict)	Folder already exists
RWS00030	HTTP code 400 (bad request)	Non sequential or non consecutive folders are not permitted

RWSI Response code	HTTP return code	Client exception / error message
RWS00031	HTTP code 404 (not found)	Form not authorised
RWS00032	HTTP code 409 (conflict)	Form does not exist
RWS00033	HTTP code 404 (not found)	Form does not exist
RWS00034	HTTP code 409 (conflict)	Form already exists
RWS00035	HTTP code 400 (bad request)	Non sequential or non consecutive forms are not permitted
RWS00036	HTTP code 409 (conflict)	Form locked
RWS00037	HTTP code 404 (not found)	Record does not exist
RWS00038	HTTP code 409 (conflict)	Record already exists
RWS00039	HTTP code 400 (bad request)	Non sequential or non consecutive records are not permitted
RWS00040	HTTP code 409 (conflict)	Record locked
RWS00041	HTTP code 404 (not found)	Field does not exist
RWS00042	HTTP code 404 (not found)	Field not authorised
RWS00043	HTTP code 409 (conflict)	Field already exists
RWS00044	HTTP code 409 (conflict)	Field locked
RWS00045	HTTP code 409 (conflict)	Hidden field/ field not visible
RWS00046	HTTP code 400 (bad request)	Picture/file upload field cannot be updated

RWSI Response code	HTTP return code	Client exception / error message
RWS00047	HTTP code 400 (bad request)	Data not in dictionary
RWS00048	HTTP code 409 (conflict)	Transaction on frozen field is not permitted
RWS00049	HTTP code 409 (conflict)	Transaction on a derived field is not permitted.
RWS00050	HTTP code 404 (not found)	Unit Dictionary does not exist
RWS00051	HTTP code 404 (not found)	Unit Dictionary entry does not exist
RWS00052	HTTP code 404 (not found)	Requested web service does not exist
RWS00053	HTTP code 404 (not found)	Cannot create more than one reusable folder at the same level
RWS00054	HTTP code 403 (forbidden)	Removing a subject is not permitted
RWS00055	HTTP code 404 (not found)	RWS URL does not exist
RWS00056	HTTP code 400 (bad request)	This is not a valid ODM 1.3 standard document
RWS00088	HTTP code 409 (conflict)	Log line restricted by max limit
RWS00089	HTTP code 403 (forbidden)	Record inactivation not authorized
RWS00091	HTTP code 400 (bad request)	
RWS00092	HTTP code 404 (not found)	
RWS00093	HTTP code 403 (forbidden)	
RWS00095	HTTP code 403 (forbidden)	Dynamic search list not supported
RWS00101	HTTP code 403 (forbidden)	Non log line form, cannot insert multiple records
RWS00102	HTTP code 403 (forbidden)	Data posted to incorrect draft URL

RWSI Response code	HTTP return code	Client exception / error message
RWS00103	HTTP code 409 (conflict)	Vendor extension mdsol:Attribute namespace pair does not exist
RWS00104	HTTP code 404 (not found)	Vendor extension mdsol:Attribute namespace pair does not exist
RWS00105	HTTP code 404 (not found)	Vendor extension mdsol:Attribute namespace does not exist
RWS00106	HTTP code 404 (not found)	Draft not found

Appendix B

Sample Code for Calling RWSI

Below are a number of snippets of code in different coding languages to illustrate how RWSI could be called to initiate an ODM data import. This includes samples for C#, Erlang, Java, Python and Ruby.

Note:
Every response from RWSI will include an HTTP return code, e.g. 200, 404. It is up to the external system calling RWSI to process this code as required. Only an HTTP return code of 200 signifies a successful import; if any other code is returned, this signifies a failure (see 'Appendix A' for more information on HTTP return codes).

Sample C# Code

The XML included in this code snippet performs a simple update.

```
Using System;
using System.Net;
using System.IO;

namespace Medidata.Integrations.RaveWebServices
{
    class ExampleClinicalDataUpdate
    {
        [STAThread]
        static void Main()
        {
            // ODM dates must be in ISO 8601 format, Rave dates must be in the field format.
            String now = DateTime.UtcNow.ToString("s");
            string subjectName = "123 ABC", siteNumber = "4567";
            string birthDate = "01 JAN 1980", gender = "MALE", race = "3";
            string server = "innovate.mdsol.com", username = "myuser", password = "mypassword";

            string odmTemplate = @"<?xml version='1.0' encoding='utf-8' ?>
```

```
            <ODM xmlns='http://www.cdisc.org/ns/odm/v1.3' ODMVersion='1.3' FileType='Transactional'
              FileOID='N/A' CreationDateTime='{0}'>
                <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
                    <SubjectData SubjectKey='{1}' TransactionType='Update'>
                        <SiteRef LocationOID='{2}'></SiteRef>
                        <StudyEventData StudyEventOID='SCREEN'>
                            <FormData FormOID='DM'>
                                <ItemGroupData ItemGroupOID='DM'>
                                    <ItemData ItemOID='BRTHDTC' Value='{3}'/>
                                    <ItemData ItemOID='SEX' Value='{4}'/>
                                    <ItemData ItemOID='RACE' Value='{5}'/>
                                </ItemGroupData>
                            </FormData>
                        </StudyEventData>
                    </SubjectData>
                </ClinicalData>
            </ODM>";

        // This will break if any of the values being inserted into the template contain un-escaped
        // special characters.
        String odm = String.Format(odmTemplate, now, subjectName, siteNumber, birthDate, gender, race);

        try
        {
            SendRequestToRave(odm, server, username, password);
            Console.WriteLine("SUCCESS");
        }
        catch (Exception exception)
        {
            Console.WriteLine("FAILURE: {0}", exception.Message);
        }

        Console.ReadLine();
    }

    static private void SendRequestToRave(string odm, string server, string username, string password)
    {
        string url = String.Format("https://{0}/RaveWebServices/WebService.aspx?PostODMClinicalData",
          server);
```

```
            HttpWebRequest request = (HttpWebRequest) WebRequest.Create(url);

            // Pass the username and password to the web service using HTTP Basic Authentication.
            Request.Credentials = new NetworkCredential(username, password);
            request.Method = “POST”;
            request.ContentType = “text/xml”;

            using (StreamWriter requestWriter = new StreamWriter(request.GetRequestStream()))
            {
                requestWriter.Write(odm);
            }

            try
            {
                using (HttpWebResponse response = (HttpWebResponse) request.GetResponse())
                {
                    WriteResponseToConsole(response.GetResponseStream());
                }
            }
            catch (WebException exception)
            {
                WriteResponseToConsole(exception.Response.GetResponseStream());
                throw exception;
            }
        }

        static private void WriteResponseToConsole(Stream responseStream)
        {
            using (StreamReader responseStreamReader = new StreamReader(responseStream))
            {
                Console.WriteLine(responseStreamReader.ReadToEnd());
                Console.WriteLine();
            }
        }
    }
}
```

Sample Erlang Code

The XML in this code snippet performs a simple update.

```
-module(rave_web_services).
-export([example1/0]).

Post_odm_clinical_data(Protocol,Domain,URL,Data,Username,Password) ->
    { ok, {{_A,HTTPResponseCode,_C}, _Headers, _Body }} = http:request(
      post,
      {
        Protocol ++ Domain ++ URL,
        [{“Authorization”, “Basic “ ++ base64:encode_to_string(Username ++ “:” ++ Password)},{“content-type”,
“text/xml”}],
        “text/xml”,
        Data
       },
      [],
      []
     ),
      [HTTPResponseCode].

Example1() ->
      Protocol=”https://”,
      Domain=”innovate.mdsol.com”,
      URL=”/Ravewebservices/WebService.aspx?PostODMClinicalData”,
      Username=”myuser”,
      Password=”mypassword”,
      SubjectKey=”100 ABC”,
      LocationOID=”MDSOL”,
      DEMO_BIRTHDT=”16 Jul 1978”,
      DEMO_SEX=”MALE”,
      DEMO_RAVE=”3”,
      DEMO_COUNTRY=”GBR”,

      ExampleODM=”<?xml version=’1.0’ encoding=’UTF-8’ ?>
<ODM xmlns=’http://www.cdisc.org/ns/odm/v1.3’ ODMVersion=’1.3’ FileType=’Transactional’ FileOID=’NA’
```

```
           CreationDateTime='2008-06-06T12:33:35'>
  <ClinicalData StudyOID='Mediflex(Dev)' MetaDataVersionOID='1'>
    <SubjectData SubjectKey='" ++ SubjectKey ++ "' TransactionType='Update'>
      <SiteRef LocationOID='" ++ LocationOID ++ "'/>
      <StudyEventData StudyEventOID='SCREEN'>
        <FormData FormOID='DM'>
          <ItemGroupData ItemGroupOID='DM'>
            <ItemData ItemOID='BRTHDTC' Value='" ++ DEMO_BIRTHDT ++ "'/>
            <ItemData ItemOID='SEX' Value='" ++ DEMO_SEX ++ "'/>
            <ItemData ItemOID='RACE' Value='" ++ DEMO_RAVE ++ "'/>
            <ItemData ItemOID='COUNTRY' Value='" ++ DEMO_COUNTRY ++ "'/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>",

      ssl:start(),
      inets:start(),
      HTTPResponseCode = post_odm_clinical_data(Protocol,Domain,URL,ExampleODM,Username,Password),
      io:fwrite("~w~n",[HTTPResponseCode]),
      inets:stop(),
      ssl:stop().
```

Sample Java Code (Apache HttpClient)

This Java example uses the Apache HttpClient class library to connect as a client to RWSI.

```
/*
      SimpleMethodClient.java

      Description:

      This simple Java class uses the Apache HttpClient class library to connect as a client to the
      Medidata Rave Web Service (Inbound) over SSL and enables an ODM 1.3.x data file to be loaded
      into Rave versions 5.6.3.
```

```
      Build Details:

            Requires the Apache class libraries,
            commons-codec-1.3.jar
            commons-httpclient-3.1.jar
            commons-logging-1.1.1.jar

            Compiled using Sun SDK EE5 update 5

            NOTE: Modify the user and host details accordingly before compiling.

      Usage:
            Simply run from the command line after successfully compiling.
            The program expects to find an ODM 1.3 data file named 'odm13data.xml' in the path.

      Copyright Medidata Solutions 2008
 */
package simpleexample.client;

import java.io.File;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.BufferedReader;

import org.apache.commons.httpclient.UsernamePasswordCredentials;
import org.apache.commons.httpclient.Credentials;
import org.apache.commons.httpclient.auth.AuthScope;
import org.apache.commons.httpclient.HttpClient;
import org.apache.commons.httpclient.HttpStatus;
import org.apache.commons.httpclient.HttpVersion;
import org.apache.commons.httpclient.methods.PostMethod;
import org.apache.commons.httpclient.methods.FileRequestEntity;
import org.apache.commons.httpclient.methods.RequestEntity;

/**
  SimpleMethodClient class
  Acts as a simple web service client to enable loading of ODM 1.3 data into Rave
**/
```

```
public class SimpleMethodClient
{
      /* Modify the username, password, hostname and odmXMLFileName accordingly */
      final static String username = “username”;
      final static String password = “password”;
      final static String hostname = “host.company.com”;
      final static String odmXMLFileName = “odm13data.xml”;

      /* Main method to connect, read the data file and upload via the web service */
      public static void main(String[] args) throws Exception
      {
            String request = “https://”+hostname+”/Ravewebservices/WebService.aspx?PostODMClinicalData”;

            HttpClient client = new HttpClient(); // Apache’s Http client
            Credentials credentials = new UsernamePasswordCredentials(username, password);

            client.getState().setCredentials(AuthScope.ANY, credentials);
            client.getState().setProxyCredentials(AuthScope.ANY, credentials); // may not be necessary

            client.getParams().setAuthenticationPreemptive(true);// send authentication details in the header

            PostMethod post = new PostMethod(request); // uses HTTP POST
            post.setDoAuthentication( true ); // must authenticate
            post.getParams().setParameter(“http.protocol.version”, HttpVersion.HTTP_1_1);

            File f = new File(odmXMLFileName);
            RequestEntity entity = new FileRequestEntity(f, “text/xml”);
            post.setRequestEntity(entity);

            try
            {
                  int statusCode = client.executeMethod(post); // send POST request with data
                  if (statusCode != HttpStatus.SC_OK)
                  { // did it work ?
                        System.err.println(“Method failed: “ + post.getStatusLine());
                  }

                  // Process the response from Rave Web Services
                  InputStream rstream = post.getResponseBodyAsStream();
```

```
                  BufferedReader br = new BufferedReader(new InputStreamReader(rstream));
                  String line;
                  while ((line = br.readLine()) != null)
                  {
                        System.out.println(line); // just display the response
                  }
                  br.close();

            }
            finally
            {
                  post.releaseConnection(); // remember to free up the connection
            }
      }
}
```

Sample Java Code (Sun JAX-WS)

This Java example uses the Sun JAX-WS class library to connect as a client to RWSI.

```
/*
      SimpleMethodClient.java

      Description:

      This simple Java class uses the Sun JAX-WS class library to connect as a client to the
      Medidata Rave Web Service (Inbound) and enables an ODM 1.3.x data file to be loaded into Rave
      versions 5.6.3.

      Build Details:

            Requires the Sun EE5 JAX-WS class libraries (in tools.jar),

            Compiled using Sun SDK EE5 update 5

            NOTE: Modify the user and host details accordingly before compiling.

      Usage:
```

```
            Simply run from the command line after successfully compiling.
            The program expects to find an ODM 1.3 data file named 'odm13data.xml' in the path.

      Copyright Medidata Solutions 2008
 */

package     simpleexample.client;

import javax.xml.ws.*;
import javax.xml.ws.http.*;
import javax.xml.ws.handler.*;
import javax.xml.namespace.Qname;
import javax.xml.transform.*;
import javax.xml.transform.stream.*;
import java.io.File;
import java.util.*;

/**
  SimpleMethodClient class
  Acts as a simple web service client to enable loading of ODM 1.3 data into Rave
**/
public class SimpleMethodClient
{
      /* Modify the username, password, hostname and odmXMLFileName accordingly */
      final static String     username = "username";
      final static String     password = "password";
      final static String     hostname = "host.company.com";
      final static String     odmXMLFileName = "odm13data.xml";

      /* Main method to connect, read the data file and upload via the web service */
      public static void main (String[] args)
      {
            Qname qname = new Qname("",   "");
            String url = "http://"+hostname+"/Ravewebservices/WebService.aspx?PostODMClinicalData";
            Service     service     = Service.create(qname);
            service.addPort(qname, HTTPBinding.HTTP_BINDING, url);
            Dispatch<Source> dispatcher   = service.createDispatch(qname,Source.class, Service.Mode.MESSAGE);
            Map<String, Object>     requestContext = dispatcher.getRequestContext();
            requestContext.put(MessageContext.HTTP_REQUEST_METHOD, "POST");
```

```
            requestContext.put(BindingProvider.USERNAME_PROPERTY, username);
            requestContext.put(BindingProvider.PASSWORD_PROPERTY, password);

            Source result =   null;
            File f = new File(odmXMLFileName);
            try
            {
                  result = dispatcher.invoke(new StreamSource(f)); // post the xml
                  printSource(result); // display the result
            }
            catch (Exception x)
            {
                  System.out.printf (“Caught Exception: %s\n”, x.toString());
                  if (result != null)     printSource(result);
            }
      }
      /*
       PrintSource
       Helper method to output the results of the post

       @param Source s the result source to read from

       */
      public static void printSource(Source s)
      {
            try
            {
                  System.out.println(“\n========================== Response Received
      ========================”);
                  TransformerFactory factory = TransformerFactory.newInstance();
                  Transformer transformer = factory.newTransformer();
                  transformer.transform(s, new StreamResult(System.out));
                  System.out.println(“\n=============================================================================”);
            }
            catch (Exception e)
            {
                  System.out.println(e);
            }
      }
```

```
}
```

Sample Python Code

The XML in this code snippet performs a simple update.

```
Import urllib2
import datetime


domain = 'innovate.mdsol.com'
webservice = '/Ravewebservices/WebService.aspx?PostODMClinicalData'
protocol = 'https://'
username = 'myuser'
password = 'mypassword'

xml='''<?xml version="1.0" encoding="utf-8" ?>
<ODM xmlns="http://www.cdisc.org/ns/odm/v1.3" ODMVersion="1.3" FileType="Transactional" FileOID="1"
CreationDateTime="%(creationdatetime)s">
  <ClinicalData StudyOID="Mediflex(Dev)" MetaDataVersionOID="1">
    <SubjectData SubjectKey="%(subjectkey)s" >
      <SiteRef LocationOID="%(locationoid)s"></SiteRef>
      <StudyEventData StudyEventOID="SCREEN">
        <FormData FormOID="DM">
          <ItemGroupData ItemGroupOID="DM">
            <ItemData ItemOID="BRTHDTC" Value="%(birthdt)s"/>
            <ItemData ItemOID="SEX" Value="%(gender)s"/>
            <ItemData ItemOID="RACE" Value="%(race)s"/>
            <ItemData ItemOID="COUNTRY" Value="%(country)s"/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>'''


data = xml % \
```

```
       {‘creationdatetime’  :   datetime.datetime.now().isoformat() ,
        ‘subjectkey’        :   ‘100 ABC’ ,
        ‘locationoid’       :   ‘MDSOL’ ,
        ‘birthdt’           :   ‘16 Jul 1978’ ,
        ‘gender’            :   ‘MALE’ ,
        ‘race’              :   ‘3’,
        ‘country’           :   ‘GBR’}


# Create a password manager and add authentication details for the Medidata Rave realm
passman = urllib2.HTTPPasswordMgr()
passman.add_password(‘Medidata Rave’, domain, username, password)
# Create an authentication handler using the password manager
authhandler = urllib2.HTTPBasicAuthHandler(passman)
# Create an opener using the authentication handler and install it
opener = urllib2.build_opener(authhandler)
urllib2.install_opener(opener)
# Create request object
req = urllib2.Request(protocol + domain + webservice,data)
# Add content type header
req.add_header(“Content-type”, “text/xml”)

# Make the request and print any errors
try:
    f = urllib2.urlopen(req)
    print f.read()
except IOError, e:
    if hasattr(e, ‘code’):
        print e.code
        print e.read()
```

Sample Ruby Code

The XML included in this code snippet performs a simple update.

```
Require ‘net/http’
require ‘net/https’
require ‘time’
```

```
def send_request_to_rave(odm, server, username, password)
  Net::HTTP.start(server, Net::HTTP.https_default_port) do |http|
    path = "/RaveWebServices/WebService.aspx?PostODMClinicalData"
    request = Net::HTTP::Post.new(path)
    request.basic_auth(username, password)
    request.content_type = "text/xml"
    request.body = odm

    response = http.request(request)
    puts response.body
    response.value
  end
end

odm_template = <<-XML
<?xml version="1.0" encoding="utf-8" ?>
<ODM xmlns="http://www.cdisc.org/ns/odm/v1.3" ODMVersion="1.3" FileType="Transactional" FileOID="N/A"
CreationDateTime="%s">
  <ClinicalData StudyOID="Beproxen(Dev)" MetaDataVersionOID="N/A">
    <SubjectData SubjectKey="%s" TransactionType="Update">
      <SiteRef LocationOID="%s"></SiteRef>
      <StudyEventData StudyEventOID="SCREEN">
        <FormData FormOID="DEMOG">
          <ItemGroupData ItemGroupOID="N/A">
            <ItemData ItemOID="DEMO_BIRTHDT" Value="%s"/>
            <ItemData ItemOID="DEMO_GENDER" Value="%s"/>
            <ItemData ItemOID="DEMO_RACE" Value="%s"/>
          </ItemGroupData>
        </FormData>
      </StudyEventData>
    </SubjectData>
  </ClinicalData>
</ODM>
XML

odm = odm_template % [
  Time.now.iso8601,
  "AK005",
```

```
  “RwsTestSite1”,
  “16 Jul 1978”,
  “Male”,
  “Asian”
]

begin
  send_request_to_rave(odm, “innovate.mdsol.com”, “myusername”, “mypassword”)
  puts “SUCCESS”
rescue
  puts “FAILURE: #{$!}”
end
```

Appendix C

List of URLs and descriptions

Metadata:

RWS URL	Method	Description
/metadata/studies	GET	Gets a list of 'Active Studies/Projects' for a Rave Url.
/metadata/libraries	GET	Gets a list of 'Active Global Library Volumes' for a Rave Url.
/metadata/studies/Mediflex/versions	GET	Gets a list of CRF Versions 'Published' to 'Mediflex' Project.
/metadata/libraries/Demo/versions	GET	Gets a list of Global Library Versions 'Published' to the Global Library (GL) 'Demo'.
/metadata/studies/Mediflex/versions/4	GET	Gets the metadata study design for CRF Version 4 for Mediflex Project.
/metadata/libraries/Demo/ ersion/43	GET	Gets the metadata study design for Global Library Version 43 for GL Demo.
/metadata/studies/Mediflex/drafts	GET	Gets the list of CRF Drafts for Mediflex Project.
/metadata/libraries/Demo/drafts	GET	Gets the list of GL Drafts for GL Demo.
/metadata/studies/Mediflex/drafts	POST	Post a CRF Draft to Mediflex Study.
/metadata/libraries/Demo/drafts	POST	Post a GL Draft to GL Demo.

Clinical Data:

RWS URL	Method	Description
/WebService.aspx?PostODMClinicalData	POST	Post clinical data to Rave.

Glossary of Terms

CDISC

Clinical **D**ata **I**nterchange **S**tandards **C**onsortium.

CDISC ODM

CDISC **O**perational **D**ata **M**odel; an XML standard for the interchange of data between clinical systems.

Data Page

A 'Form' in Rave terms. See 'Form'.

Data Point

A 'Field' in Rave terms. A data point represents the actual data that is entered by a user for a specific subject in a study. It is a single atomic unit of clinical information stored in the system.

Field

A 'Data Point' in EDC terms. See 'Data Point'.

Folder

An 'Instance' in EDC terms. See 'Instance'.

Form

A 'Data Page' in EDC terms. A Form is a documented set of data entry items. It can be literal (a screen or paper that a user uses to type or write data into) or conceptual (data loaded from another system via a software interface).

HTTPS

Hyper**T**ext **T**ransfer **P**rotocol over **S**ecure Socket Layer or HTTPS is a URI scheme used to indicate a secure HTTP connection. It is syntactically identical to the http:// scheme normally used for accessing resources using HTTP.

Instance

A 'folder' in Rave. An Instance contains subject data for a particular folder; or contains Records (data for specific Forms). The Instance has a range of dates that specifies when data entry is permissible, when not permissible, when overdue, etc.

Rave Account

Login ID and password used to connect to Rave and Rave database.

Rave Folder Path

A unique way for indentifying a Rave instance within a multi level folder structure.

Record

A Record contains subject data associated with a specific form. Also termed a LogLine record in EDC.

Request

A Request is a set of commands from one system to another, these commands contains for e.g. content for export, destination server name, username, password, database connection string and support email address. In this case, this also contains the ODM XML document.

Response

This is the response received from Rave via RWSI when a request has been submitted to add, update or remove clinical data. This will include information concerning the success or failure of the request.

RWSI

Rave **W**eb **S**ervices **I**nbound v1.0.

Support

The support team receiving the RWSI failure messages by email.

Transaction

This ensures the integrity of any changes to the Rave database. RWSI only commits a transaction if all data in the request is correct, if an error occurs during processing, the RWSI rolls-back the transaction and no changes are made to the database.

XML

E**x**tensible **M**arkup **L**anguage

Xpath

The syntax for addressing parts of an XML document
(e.g. /study12/folder34/form56/field78[@OID]).

Revision History

Version	Date	Description of Changes
1.0	Jun 2008	Original Release
2.0	May 2009	Incremented for v1.0.1 patch.
3.0	Dec 2009	Release for v1.0.2 patch.